arXiv:2606.25603v1 [stat.CO] 24 Jun 2026

Resampling in conditional SMC algorithms

Axel Finke* Adam M. Johansen† Anthony Lee‡
Lawrence M. Murray§

June 25, 2026

Conditional sequential Monte Carlo (CSMC) algorithms arise in particle Markov chain Monte Carlo and a number of related settings. As in standard sequential Monte Carlo (SMC) algorithms, it is possible to employ a number of approaches to resampling within CSMC, but some additional care is required to arrive at a valid algorithm. We present a simple framework for implementing valid SMC and CSMC algorithms which (a) covers most known resampling schemes including those with a complicated dependence structure like systematic resampling, but also adaptive resampling, and even more 'exotic' schemes like a version of chopthin resampling; (b) explains how to implement conditional analogues of these and other well known resampling schemes without randomly permuting/shifting the order of the ancestor indices; (c) requires only very weak assumptions which include neither (marginal) 'unbiasedness' nor exchangeability.

1 Introduction

1.1 Resampling

Sequential Monte Carlo (SMC) algorithms (also termed *particle filters*) are a class of Monte Carlo methods for approximating (sequences of) probability measures and their

*School of Mathematics, Statistics and Physics, Newcastle University, Newcastle upon Tyne, UK. axel.finke@ncl.ac.uk. ORCID: 0000-0002-8379-3012
†Department of Statistics, University of Warwick, Coventry, UK. a.m.johansen@warwick.ac.uk. ORCID: 0000-0002-3531-7628
‡School of Mathematics, University of Bristol, Bristol, UK. anthony.lee@bristol.ac.uk. ORCID: 0000-0001-7765-0616
§Independent researcher. lawrence@indii.org. ORCID: 0000-0002-6567-6015

normalising constants such as marginal likelihoods (see, e.g., Chopin and Papaspiliopoulos, 2020, and references therein). The closely related *conditional sequential Monte Carlo (CSMC)* algorithm induces a Markov chain which can be used to efficiently approximate the joint posterior distribution of all the latent states in state-space models (a.k.a., the *joint smoothing distribution*), and more generally, due to its ability to exploit the 'decorrelation-over-time' structure of the target distribution (Andrieu et al., 2010; Lee et al., 2020; Karjalainen et al., 2025). CSMC methods are also at the core of more complex schemes, e.g., for unbiased estimation (Jacob et al., 2019; van den Boom et al., 2022; Shestopaloff and Doucet, 2019) or diffusion models (Corenflos et al., 2024; Stanton and Beskos, 2024).

A key ingredient in SMC algorithms is resampling and CSMC algorithms rely on a related *conditional* resampling mechanism. Numerous resampling schemes have been proposed, e.g., multinomial resampling (Rubin, 1987), stratified resampling (Kitagawa, 1996), systematic resampling (Carpenter et al., 1999), residual resampling (Liu and Chen, 1998; Douc et al., 2005), adaptive resampling Liu and Chen (1995), killing resampling (Del Moral, 2013; Chopin et al., 2022), Metropolis resampling (Murray et al., 2016), Hilbert-curve resampling (Gerber and Chopin, 2015), chopthin resampling (Gandy and Lau, 2015), as well as further sophisticated resampling schemes in Gerber et al. (2019); Chopin et al. (2022). For practitioners, it is therefore important to have verifiable conditions which guarantee that their chosen resampling scheme leads to SMC or CSMC algorithms that are valid. In this work, we introduce a perspective which allows us to view most of these algorithms within a common framework which provides such guarantees.

1.2 Existing frameworks

We will consider 'validity' of SMC and CSMC algorithms only in the sense that the former yield unbiased estimates of normalising constants (e.g., marginal likelihoods) and that the latter induce Markov kernels which leave the target distribution (e.g., joint smoothing distribution) invariant. We do not analyse the efficiency (e.g. convergence rates) associated with different resampling schemes. For the latter, we refer the reader to Douc et al. (2005); Whiteley et al. (2016); Gerber et al. (2019); Webber (2019); Chopin et al. (2022); Li et al. (2022).

When homogeneous (i.e., equal) post-resampling weights are sought, a common requirement used to ensure validity (in the above-mentioned sense) is the conventional *'unbiasedness'* condition from Crisan et al. (1999) which says that the expected number of descendants of each particle should be proportional to its weight.

For CSMC algorithms, the seminal work of Andrieu et al. (2010) additionally required that resampling schemes should be exchangeable. They also noted that exchangeability can always be ensured by randomly permuting the order of the ancestor indices after resampling. This idea was used in Chopin and Singh (2015) to derive a conditional version of residual-multinomial resampling. However, such random permutations can incur a non-negligible computational cost, especially on parallel architectures (Murray et al., 2016), and the framework from Andrieu et al. (2010) does not cover adaptive

resampling.

Alternatively, Chopin and Singh (2015); Karppinen et al. (2023) imposed a so-called *marginal 'unbiasedness'* condition which implies the conventional 'unbiasedness' condition but does not require exchangeability. For some non-exchangeable resampling schemes (i.e., systematic resampling and killing resampling) the authors were thus able to derive a valid CSMC algorithm by applying a random shift to the order of the ancestor indices after resampling. However, the random shift again incurs additional computational cost and their framework does not cover adaptive resampling.

It has long been known that the conventional 'unbiasedness' condition is not necessary and can be dropped at the expense of non-uniform post-resampling weights (e.g., Liu, 2001). Chen (2010) justified this strategy as changing the sequence of intermediate distributions of the SMC or CSMC algorithm. However, this justification does not apply to non-exchangeable resampling schemes such as the partial resampling scheme from Martino et al. (2016) or to the optimal finite-state resampling scheme from Fearnhead and Clifford (2003).

Finally, Whiteley et al. (2016); Huggins and Roy (2019) proved the validity of SMC and CSMC algorithms for (adaptive) resampling schemes that fit into their αSMC framework. However, this framework's resampling-matrix representation only exists for resampling schemes in which the ancestor indices are conditionally independent and thus does not apply to the widely used systematic resampling scheme nor to Metropolis resampling or Hilbert-curve resampling.

1.3 Contributions

We show how to construct valid SMC and CSMC algorithms based on a wide range of resampling schemes and in particular:

1. In Section 2, we present a simple framework for resampling which covers most known resampling schemes and their conditional versions as special cases, including those with a complicated dependence structure like (conditional) systematic resampling and those which do not satisfy the conventional 'unbiasedness' condition from Crisan et al. (1999) such as partial resampling (Martino et al., 2016), optimal finite-state resampling scheme (Fearnhead and Clifford, 2003) or a version of chopthin resampling (Gandy and Lau, 2015).

2. Key to our framework is the introduction of an *index distribution* $\lambda(\cdot\,|n)$ which governs the law of the particle index of the descendant of particle n on a suitably extended space. By making the rôle and the (potentially non-uniform) choice of this index distribution explicit, our work automatically provides conditional versions of any resampling scheme within our framework. For example, our work yields a novel conditional version of chopthin resampling. Further, our work makes it possible to implement conditional versions of non-exchangeable resampling schemes without having to randomly permute or shift the order of the ancestor indices.

3. The assumptions of our framework are weaker than those imposed in the existing frameworks from Andrieu et al. (2010); Chopin and Singh (2015); Karppinen et al. (2023); Whiteley et al. (2016); Huggins and Roy (2019), requiring only a mild *persistence* condition (Definition 8) which ensures that the post-resampling weights cannot all be simultaneously zero and a *proper weighting* assumption (Definition 5). Additionally, we specify *balanced* index distribution (Definition 6) which automatically ensures proper weighting. Section 3 shows that most known resampling schemes implicitly use a balanced index distribution.

4. In Section 4, we prove that our framework makes it simple to guarantee (a) the unbiasedness of normalising-constant estimates generated by SMC algorithms; (b) the validity of the corresponding CSMC algorithm (with backward/ancestor sampling), for a wide range of resampling schemes—including adaptive/systematic resampling. In Appendix A, we further demonstrate that naïve implementations of conditional resampling sometimes used by practitioners can bias the Markov chain induced by the CSMC algorithm and may even worsen its autocorrelation.

1.4 Notation

We let $[n] \coloneqq \{1, \dots, n\}$, for any $n \in \mathbb{N} \coloneqq \{1, 2, \dots\}$ and use the convention $\frac{0}{0} \coloneqq 0$. Throughout this work, we assume some background probability space $(\Omega, \mathcal{A}, \mathbb{P})$; $\mathcal{B}(\mathsf{X})$ denotes the Borel σ-algebra on some topological space X. For any measure space $(\mathsf{X}, \mathcal{X}, \mu)$, where μ may be random, we write $\mu(f) \coloneqq \int_{\mathsf{X}} f(x)\mu(\mathrm{d}x)$, for μ-integrable functions $f \colon \mathsf{X} \to \mathbb{R}$. Abusing notation, we frequently use the same symbol for measures and their densities (w.r.t. a suitable dominating measure, e.g., Lebesgue or counting measure); $\mathrm{vol}(A)$ is the volume (i.e., Lebesgue measure) of measurable $A \in \mathbb{R}^d$; $\mathrm{Cat}(p_{1:K})$ denotes the categorical distribution on $[K]$ with probability vector $p_{1:K} \in [0,1]^K$ such that $\sum_{k=1}^K p_k = 1$. For a suitable set B, Unif_B is the (discrete or continuous) uniform distribution on B; $\delta_x(B) = \mathbf{1}\{x \in B\}$ is the point mass/Dirac measure at x; if $B = \{b\}$ is a singleton, we write $\delta_x(\{b\}) =: \delta_x(b)$.

2 Framework for the validity of resampling schemes

2.1 Resampling

2.1.1 Goal of resampling

In this section, we present a generic framework for (conditional) resampling. We acknowledge that this necessitates a level of abstraction whose utility will become fully apparent in Section 3, where we illustrate how most known resampling schemes fit into our framework. Due to the notational burden of SMC and CSMC algorithms, we defer these methods to Section 4 and treat (conditional) resampling in a simpler setting until then.

Hereafter, assume some measurable space $(\mathsf{X}, \mathcal{B}(\mathsf{X}))$ where X is Polish, e.g., $\mathsf{X} = \mathbb{R}^d$. Consider a (possibly random) weighted sample $(\mathbf{X}, \mathbf{W}) \coloneqq (X^{1:N}, W^{1:N})$, for some $N \in \mathbb{N}$. Here, for all $n \in [N]$, X^n is a sample point ('particle') that takes values in X and W^n is a non-negative weight, such that $\sum_{n=1}^N W^n = 1$. Such a weighted sample can be associated with a probability measure on $(\mathsf{X}, \mathcal{B}(\mathsf{X}))$:

$$\hat{\pi} \coloneqq \sum_{n=1}^{N} W^n \delta_{x^n}.$$

Resampling is a procedure that replaces $(\mathbf{X}, \mathbf{W})$ with random variables $(\widetilde{\mathbf{X}}, \widetilde{\mathbf{w}}) \coloneqq (\widetilde{X}^{1:M}, \widetilde{w}^{1:M})$, for some $M \in \mathbb{N}$, which can be associated with the random measure:

$$\tilde{\pi} \coloneqq \sum_{m=1}^{M} \widetilde{w}^m \delta_{\widetilde{X}^m}.$$

A typical goal of resampling is to ensure that $\tilde{\pi}$ is close to $\hat{\pi}$ with $\widetilde{w}^{1:M}$ more homogeneous than $W^{1:N}$, in a suitable sense, whilst maintaining *unbiasedness* i.e., that $\mathbb{E}[\tilde{\pi}(f)|\mathcal{F}] = \hat{\pi}(f)$, for any suitably integrable test function $f \colon \mathsf{X} \to \mathbb{R}$ where $\mathcal{F} \coloneqq \sigma(\mathbf{X}, \mathbf{W})$ is the σ-algebra generated by $(\mathbf{X}, \mathbf{W})$.

We will refer to the $\widetilde{w}^m \geq 0$ as post-resampling 'weights', though it is possible that $\sum_{m=1}^M \widetilde{w}^m \neq 1$ (equality holds for most of the commonly used resampling schemes and must hold at least in expectation, i.e., $\mathbb{E}[\sum_{m=1}^M \widetilde{w}^m|\mathcal{F}] = 1$, almost surely, for resampling schemes that are unbiased).

Example 1 (sampling–importance resampling). *Weighted samples* $(\mathbf{X}, \mathbf{W})$ *arise in* self-normalised importance sampling (SNIS). *Here, we are interested in expectations of the form* $\pi(f) = \int_{\mathsf{X}} f(x)\pi(\mathrm{d}x)$, *for a probability measure* $\pi(\mathrm{d}x) \propto G(x)Q(\mathrm{d}x)$ *whose normalising constant is typically intractable;* Q *is a probability measure on* X, $G \colon \mathsf{X} \to [0, \infty)$ *is some non-negative* Q*-integrable function and* $f \colon \mathsf{X} \to \mathbb{R}$ *is some* π*-integrable function. SNIS then estimates* $\pi(f)$ *as* $\hat{\pi}(f) = \sum_{n=1}^N W^n f(X^n)$, *for* $X^1, \ldots, X^N \overset{\text{IID}}{\sim} Q$ *and (self-normalised) importance weights* $W^n \coloneqq G(X^n)/\sum_{l=1}^N G(X^l)$. *After resampling, expectations* $\pi(f)$ *can again be estimated by* $\tilde{\pi}(f)$. *This is known as the* sampling–importance resampling (SIR) *estimator (Rubin, 1987). The unbiasedness condition then implies that the SIR estimate does not incur additional bias compared to the original SNIS estimate.*

2.1.2 Specific class of resampling schemes treated in this work

Although resampling is very general and imposes almost no constraints on how $(\widetilde{\mathbf{X}}, \widetilde{\mathbf{w}})$ is produced, we focus in the sequel on the a class of resampling schemes which specify the resampled particles as $\widetilde{X}^m \coloneqq X^{A^m}$, where A^m is an *ancestor index* taking a value in $[N]$. Definition 1 gives a sampling distribution for $\mathbf{A} \coloneqq A^{1:M}$ and for some associated optional collection of auxiliary variables U used in the simulation of $\mathbf{A}$. The latter take values in some set U; if no such auxiliary variables are used, we set $\mathsf{U} \coloneqq \emptyset$ and suppress U and u from the notation.

Definition 1 (resampling distribution). *A distribution* $\rho(\mathbf{a}, \mathrm{d}u) = \mathbb{P}(\mathbf{A} = \mathbf{a}, U \in \mathrm{d}u)$ *on* $[N]^M \times \mathsf{U}$ *is called a a* resampling distribution *if, for any* $n \in [N]$, $W^n > 0 \implies \sum_{m=1}^M \mathbb{P}(A^m = n) > 0$.

For any $m \in [M]$, we define the following notation: $\rho^m(a, \mathrm{d}u) \coloneqq \mathbb{P}(A^m = a, U \in \mathrm{d}u)$ is the marginal distribution of (A^m, U) under ρ; $\rho^m(a) \coloneqq \mathbb{P}(A^m = a)$ is the marginal distribution of A^m under ρ; $\rho^{-m}(\mathbf{a}^{-m}|a, u) \coloneqq \mathbb{P}(\mathbf{A}^{-m} = \mathbf{a}^{-m}|A^m = a, U = u)$ (assuming $M > 1$) is the full conditional distribution of $\mathbf{A}^{-m} \coloneqq (A^1, \ldots, A^{m-1}, A^{m+1}, \ldots, A^M)$ under ρ.

Example 2 (toy resampling scheme). *Assume* $N = M = 2$ *and* $\mathsf{U} = \emptyset$ *and* $W^1 \in (0, 1)$ *so that* $W^2 = 1 - W^1 \in (0, 1)$. *For some* $\alpha, \beta \in [0, 1]$, *let* $\rho^1 = \alpha\delta_1 + (1 - \alpha)\delta_2$ *and* $\rho^2 = \beta\delta_1 + (1 - \beta)\delta_2$ *be the marginal distributions of* ρ. *Then* ρ *is a resampling distribution if* $0 < \alpha + \beta < 2$. *Because then each particle is chosen as an ancestor with positive probability – for any joint distribution* ρ *with the above-mentioned marginals.*

Definition 2 (index distribution). *Given* $n \in [N]$, *a conditional distribution* $\lambda(m, \mathrm{d}u|n)$ *on* $[M] \times \mathsf{U}$ *is called an* index distribution.

The key rôle of index distributions for validity of (conditional) resampling is the focus of this work.

Definition 3 (resampling scheme). *A* resampling scheme *is a tuple* $\langle \rho, \lambda \rangle$ *consisting of (1) a resampling distribution* ρ *and (2) a family of index distributions* $\lambda = (\lambda(\,\cdot\,|n))_{n \in [N]}$, *which generates* $(\widetilde{\mathbf{X}}, \widetilde{\mathbf{w}})$ *as summarised in Algorithm 1, where*

$$\phi(m, n, u) \coloneqq \frac{W^n \lambda(m, u|n)}{M^{-1} \rho^m(n, u)}. \tag{1}$$

Algorithm 1 Resampling

1: Sample $(\mathbf{A}, U) = (A^{1:M}, U) \sim \rho(\mathbf{a}, \mathrm{d}u)$.
2: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq \phi(m, A^m, U)/M$, for $m \in [M]$.

When necessary, we will make the dependence of ρ, ρ^m, $\rho^{-m}(\,\cdot\,; n)$, $\lambda(\,\cdot\,|n)$ and ϕ on the weights $\mathbf{W} = W^{1:N}$ explicit in the notation by writing $\rho = \rho(\,\cdot\,; \mathbf{W})$ and $\rho^m = \rho^m(\,\cdot\,; \mathbf{W})$, $\rho^{-m}(\,\cdot\,|n; \mathbf{W})$, $\lambda(\,\cdot\,|n; \mathbf{W})$ and $\phi(\,\cdot\,; \mathbf{W})$; and, in principle, these could also depend on $\mathbf{X}$. The dependence of these quantities (and also of U) on N and M will always be clear from the context.

2.1.3 Sufficient condition for unbiasedness

A frequently-imposed requirement on resampling schemes is that $\tilde{\pi}$ is an unbiased estimate of $\hat{\pi}$ in the following sense, where we remember that $\mathcal{F} = \sigma(\mathbf{X}, \mathbf{W})$ is the σ-algebra generated by $(\mathbf{X}, \mathbf{W})$.

Definition 4 (unbiasedness). *A resampling scheme* $\langle \rho, \lambda \rangle$ *is* unbiased *if*

$$\mathbb{E}[\tilde{\pi}(f)|\mathcal{F}] = \hat{\pi}(f),$$

almost surely, for any bounded continuous functions $f \colon \mathsf{X} \to \mathbb{R}$.

Definition 5 (proper weighting). *A resampling scheme* $\langle \rho, \lambda \rangle$ *is called* properly weighted *if* (1) *is well defined, i.e., if for any* $(m, n, u) \in [M] \times [N] \times \mathsf{U}$, $\rho^m(n, u) = 0 \implies W^n \lambda(m, u|n) = 0$.

Example 3 (toy resampling scheme, continued). *Consider the following families of index distributions* $\lambda_i \coloneqq (\lambda_i(\,\cdot\,|n))_{n\in[N]}$, *for* $i \in [2]$*:*

1. $\lambda_1(\,\cdot\,|1) \propto \alpha\delta_1 + \beta\delta_2$; $\lambda_1(\,\cdot\,|2) \propto (1-\alpha)\delta_1 + (1-\beta)\delta_2$;
2. $\lambda_2(\,\cdot\,|1) = \lambda_2(\,\cdot\,|2) \coloneqq \mathrm{Unif}_{[2]}$.

If $\alpha, \beta \in (0, 1)$ *then* $\langle \rho, \lambda_1 \rangle$ *and* $\langle \rho, \lambda_2 \rangle$ *are both properly weighted because the numerator and denominator in* (1) *is are strictly positive for any* $(m, n) \in [2]^2$. *If* $\alpha = \beta = 1$ *then* $\langle \rho, \lambda_1 \rangle$ *is still properly weighted because* $\rho^m(n) = \lambda_1(m|n) = \mathbf{1}\{m = n\}$ *so that the denominator in* (1) *is zero only if the numerator is zero. However, in this case, However,* $\langle \rho, \lambda_2 \rangle$ *is not properly weighted because for* $(m, n) \in [2]^2$ *with* $m \neq n$*, the denominator in* (1) *is zero while the numerator remains strictly positive. Put differently, the second choice of index distribution allows for assignments incompatible with the resampling distribution causing* ϕ *to be undefined (albeit on a set which is null w.r.t.* ρ*).*

The following is the main result of this section. It shows that proper weighting is sufficient for unbiasedness. Motivated by the fact that $\hat{\pi}(f) = \mathbb{E}_{A\sim\mathrm{Cat}(\mathbf{W})}[f(X^A)|\mathcal{F}]$, the proof interprets resampling as importance sampling (with one sample point) targetting $\mathrm{Cat}(\mathbf{W})$. More precisely, we perform importance sampling on the extended space that includes $(K, A, \mathbf{A}, U)$, where $K \in [M]$ is the index of a single ancestor index and $A \in [N]$ is its value which is almost surely equal to A^K. Proper weighting then guarantees that the target distribution of this importance-sampling construction on $[M]\times[N]\times[N]^M\times\mathsf{U}$:

$$\mu(k, a, \mathbf{a}, \mathrm{d}u) \coloneqq \mathrm{Cat}(a; \mathbf{W})\lambda(k, \mathrm{d}u|a)\delta_a(a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k, u), \tag{2}$$

(which admits $\mathrm{Cat}(\mathbf{W})$ as a marginal) is absolutely continuous w.r.t. the sampling distribution

$$q(k, a, \mathbf{a}, \mathrm{d}u) \coloneqq \rho(\mathbf{a}, \mathrm{d}u)\,\mathrm{Unif}_{[M]}(k)\delta_{a^k}(a). \tag{3}$$

Proposition 1. *A properly weighted resampling scheme is unbiased.*

Proof. For the moment, assume that $M > 1$. By Definition 5, $\mu(a, k, \mathbf{a}, \mathrm{d}u) \ll q(k, a, \mathbf{a}, \mathrm{d}u)$, so that the following change of measure—with Radon–Nikodym derivative $[\mathrm{d}\mu/\mathrm{d}q](k, a, \mathbf{a}, u) = \phi(k, a, u)$ given in (1)—is valid, where $\mathbb{E}_q[\,\cdot\,|\mathcal{F}]$ is shorthand for $\mathbb{E}_{(K,A,\mathbf{A},U)\sim q}[\,\cdot\,|\mathcal{F}]$:

$$\begin{aligned}
\hat{\pi}(f) = \sum_{n=1}^{N} W^n f(X^n) &= \mathbb{E}_{A\sim\mathrm{Cat}(\mathbf{W})}[f(X^A)|\mathcal{F}] = \mathbb{E}_{(K,A,\mathbf{A},U)\sim\mu}[f(X^A)|\mathcal{F}] \\
&= \mathbb{E}_q\left[\frac{\mathrm{d}\mu}{\mathrm{d}q}(K, A, \mathbf{A}, U) f(X^A) \,\middle|\, \mathcal{F}\right] \\
&= \mathbb{E}_q\left[\mathbb{E}_q\left(\frac{\mathrm{d}\mu}{\mathrm{d}q}(K, A, \mathbf{A}, U) f(X^A) \,\middle|\, \sigma(\mathbf{A}, U) \vee \mathcal{F}\right) \,\middle|\, \mathcal{F}\right] \\
&= \mathbb{E}_q\left[\frac{1}{M}\sum_{m=1}^{M} \phi(m, A^m, U) f(X^{A^m}) \,\middle|\, \mathcal{F}\right] = \mathbb{E}[\tilde{\pi}(f)|\mathcal{F}].
\end{aligned}$$

For $M = 1$, the proof is analogous except that we omit $\rho^{-k}(\mathbf{a}^{-k}|a^k, u)$ in (2). □

2.1.4 Standard choice of index distribution

The proper weighting definition leaves a degree of freedom in choosing the family of index distributions. The following class of resampling schemes provides a standard choice which automatically guarantees proper weighting by taking $\lambda(m, \mathrm{d}u|n) \propto \rho^m(n, \mathrm{d}u)$.

Definition 6 (balance). *A resampling scheme $\langle \rho, \lambda \rangle$ is* balanced *if for any $(m, n, u) \in [M] \times [N] \times \mathsf{U}$:*

$$W^n > 0 \quad \Longrightarrow \quad \lambda(m, \mathrm{d}u|n) = \frac{\rho^m(n, \mathrm{d}u)}{\sum_{l=1}^{M} \rho^l(n)}. \tag{4}$$

The name *balanced* is motivated by the fact that (4) is essentially the *balance heuristic* for multiple importance sampling from Veach and Guibas (1995) (see also Medina-Aguayo and Everitt (2019) for further discussions on the balance heuristic and its relationship with extended state-space justifications of importance sampling similar to (2)–(3)).

Within the importance-sampling construction used in the proof of Proposition 1, (4) implies that $\lambda(m, \mathrm{d}u|n) = q(m, \mathrm{d}u|n)$ (whenever this conditional distribution is well defined). Balanced resampling schemes have that $\phi(m, n, u) = MW^n / \sum_{l=1}^{M} \rho^l(n)$ is almost-surely constant in (m, u), i.e., in the proof of Proposition 1, they perform importance sampling on a smaller space which does not include (K, U).

As Section 3 will demonstrate, most known resampling schemes are balanced, i.e., they implicitly use the index distribution from (4). The definition of proper weighting implies the following result.

Proposition 2. *A balanced resampling scheme is properly weighted.*

The following example illustrates that using a balanced resampling scheme not only guarantees proper weighting but also ensures that the conditional variance of resampling is robust to single small marginal probabilities $\rho^m(A^m)$ under the resampling distribution.

Example 4 (toy resampling scheme, continued). *Assume that $\alpha, \beta \in (0,1)$. Then as discussed above, $\langle \rho, \lambda_1 \rangle$ and $\langle \rho, \lambda_2 \rangle$ are both properly weighted. However, $\langle \rho, \lambda_1 \rangle$ is balanced while $\langle \rho, \lambda_2 \rangle$ is not (unless $\alpha = \beta = 1/2$). For simplicity. let $f \equiv 1$ and fix $\beta, W^1 \in (0,1)$. Then:*

1. *the balanced scheme $\langle \rho, \lambda_1 \rangle$ has $\sup_{\alpha \in (0,1)} \mathrm{var}[\tilde{\pi}(f)|\mathcal{F}] < \infty$;*
2. *the imbalanced scheme $\langle \rho, \lambda_2 \rangle$ has $\lim_{\alpha \downarrow 0} \mathrm{var}[\tilde{\pi}(f)|\mathcal{F}] = \lim_{\alpha \uparrow 1} \mathrm{var}[\tilde{\pi}(f)|\mathcal{F}] = \infty$.*

Intuitively, λ_1 is 'adapted' to the proposal distribution in the importance-sampling construction. For instance, as $\alpha \downarrow 0$, the probability $\rho^1(1)$ vanishes. The family λ_1 accounts for this since $\alpha \downarrow 0$ implies $\lambda_1(1|1) \propto \rho^1(1) \downarrow 0$. In contrast, λ_2 fixes $\lambda_2(1|1) = 1/2$.

As noted above, a frequent goal of resampling is to make the resampled weights $\widetilde{w}^{1:M}$ more homogeneous than $W^{1:N}$. The following class of resampling schemes achieves $\widetilde{w}^m = 1/M$, for all $m \in [M]$.

Definition 7 (weight-homogeneous resampling scheme). *A balanced resampling scheme $\langle \rho, \lambda \rangle$ is called* weight-homogeneous *if $\phi \equiv 1$ (almost surely), i.e., if for all $n \in [N]$:*

$$\sum_{m=1}^{M} \rho^m(n) = MW^n. \tag{5}$$

Since achieving $\widetilde{w}^m = 1/M$ is frequently the goal of resampling, (5) is itself considered *the* unbiasedness condition (Crisan et al., 1999). We will thus call (5) the conventional 'unbiasedness' condition.

Example 5 (toy resampling scheme, continued). *Simple algebra shows that the balanced resampling scheme $\langle \rho, \lambda_1 \rangle$ is weight-homogeneous if $\alpha + \beta = 2W^1$.*

Definition 8 (persistence). *Let $B_+ := \{(\mathbf{a}, u) \in [N]^N \times \mathsf{U} | \sum_{m=1}^{M} \phi(m, a^m, u) > 0\}$ be the event that the sum of the post-resampling weights under the resampling scheme $\langle \rho, \lambda \rangle$ is strictly positive. Then $\langle \rho, \lambda \rangle$ is called* persistent *if $\rho(B_+; W^{1:N}) = 1$, for any $W^1, \ldots, W^N \geq 0$ with $\sum_{n=1}^{N} W^n = 1$.*

Example 6 (toy resampling scheme, continued). *If $\alpha, \beta \in (0,1)$ then both $\langle \rho, \lambda_1 \rangle$ and $\langle \rho, \lambda_2 \rangle$ are persistent. However, the properly weighted resampling scheme $\langle \rho, \lambda_3 \rangle$ given by $\lambda_3(\cdot|n) = \delta_n$, for $n \in [2]$, is not because $A^{1:2} = (2,1)$ and hence $\widetilde{w}^1 = \widetilde{w}^2 = 0$ has positive probability under $\langle \rho, \lambda_3 \rangle$*

Persistence guarantees that the post-resampling weights $\widetilde{w}^m$ can (almost surely) be self-normalised so that another round of resampling could be applied to $(\widetilde{X}^{1:M}, \widetilde{W}^{1:M})$, where $\widetilde{W}^m := \widetilde{w}^m / \sum_{l=1}^{M} \widetilde{w}^l$. The following sufficient condition for persistence is immediate from the definition.

Proposition 3. *A balanced resampling scheme $\langle \rho, \lambda \rangle$ is persistent if for all $n \in [N]$: $\sum_{m=1}^{M} \rho^m(n) > 0 \Longrightarrow W^n > 0$.*

Remark 1. *The sufficient condition for persistence given in Proposition 3 holds immediately if the resampling scheme $\langle \rho, \lambda \rangle$ is weight-homogeneous.*

2.2 Conditional resampling

We will now turn to *conditional* resampling. This is a key ingredient in CSMC methods but we present it here without reference to the latter (the extension to CSMC algorithms will be discussed in Section 4.2). Admittedly, this makes our discussion below somewhat abstract. However, we believe that abstraction is justified by the fact that doing so avoids the notational burden incurred by CSMC methods.

Throughout this section, we assume that $M \geq 2$ and that the resampling scheme $\langle \rho, \lambda \rangle$ is persistent (so that the post-resampling 'weights' can be normalised). Given some $a \in [N]$ with $W^a > 0$, conditional resampling (Algorithm 2) forms a (possibly random) probability measure

$$\tilde{\pi}^a := \sum_{m=1}^{M} \frac{\widetilde{w}^m}{\sum_{l=1}^{M} \widetilde{w}^l} \delta_{\widetilde{X}^m},$$

in such a way that the support of $\tilde{\pi}^a$ includes X^a.

Algorithm 2 conditional resampling

Require: $a \in [N]$ with $W^a > 0$.

1: Sample $(K, U, \mathbf{A}) \sim \lambda(k, u|a)\delta_a(a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k, u)$.

2: Set $\widetilde{X}^m := X^{A^m}$ as well as $\widetilde{w}^m := \phi(m, A^m, U)/M$, for $m \in [M]$.

The following requirement says that if we first sample $A = a$ with probability W^a and then perform conditional resampling, $\tilde{\pi}^a$ should again induce a random measure that is unbiased for $\hat{\pi}$.

Definition 9 (invariance). *A persistent resampling scheme $\langle \rho, \lambda \rangle$ is* invariant *if*

$$\sum_{a=1}^{N} W^a \, \mathbb{E}[\tilde{\pi}^a(f)|\mathcal{F}] = \hat{\pi}(f),$$

almost surely, for all bounded continuous functions $f \colon \mathsf{X} \to \mathbb{R}$.

The terminology 'invariance' is motivated by the fact that Definition 9 holds if executing Algorithm 2 and then returning the ancestor index A^L with probability proportional to $\widetilde{w}^L$ induces a $\mathrm{Cat}(\mathbf{W})$-invariant Markov kernel. The main result of this section is the following.

Proposition 4 (proper weighting implies invariance). *A properly weighted and persistent resampling scheme is invariant.*

Proof. We extend the distribution μ from the proof of Proposition 1 so that it represents the law of first sampling $A \sim \mathrm{Cat}(\mathbf{W})$, then running Algorithm 2 (with $A = a$) and finally selecting the Lth ancestor index with probability proportional to the Lth post-resampling 'weight' (and calling it B).

$$\mu(k, a, \mathbf{a}, \mathrm{d}u, b, l) := \mathrm{Cat}(a; \mathbf{W})\lambda(k, \mathrm{d}u|a)\delta_a(a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k, u)\frac{\phi(l, a^l, u)}{\sum_{m=1}^{M} \phi(m, a^m, u)}\delta_{a^l}(b).$$

By persistence and proper weighting, this distribution is well defined. Simple algebra verifies the symmetry $\mu(k, a, \mathbf{a}, \mathrm{d}u, b, l) = \mu(l, b, \mathbf{a}, \mathrm{d}u, a, k)$ which in turn implies

$$\sum_{a=1}^{N} W^a \, \mathbb{E}[\tilde{\pi}^a(f)|\mathcal{F}] = \mathbb{E}_\mu[f(X^B)|\mathcal{F}] = \mathbb{E}_\mu[f(X^A)|\mathcal{F}] = \hat{\pi}(f),$$

where $\mathbb{E}_\mu[\,\cdot\,|\mathcal{F}]$ denotes expectation w.r.t. $(K, A, \mathbf{A}, U, B, L) \sim \mu$. □

2.3 Implementation of conditional resampling with fixed reference index

In practice, conditional resampling (within CSMC algorithms) is often implemented by placing the ancestor index a in a fixed location $K = 1$ or $K = M$, say, in Line 1 of Algorithm 2. While this strategy is generally invalid (invariance is lost), the next proposition (proved in Appendix C) shows that it *is* valid when the resampling distribution is symmetric in the following sense.

Definition 10 (reindexability). *A resampling scheme $\langle \rho, \lambda \rangle$ is called* reindexable *if $\mathbb{E}_{k,a}[f(\mathbf{A})|\mathcal{F}] = \mathbb{E}_{1,a}[f(\mathbf{A})|\mathcal{F}]$, for any $a \in [N]$ with $W^a > 0$, any $k \in [M]$ and any function $f\colon [N]^M \to \mathbb{R}$ which is symmetric in its M arguments, where $\mathbb{E}_{k,a}$ denotes expectation w.r.t. $\mathbf{A} \sim \delta_a(a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k, u)$.*

Proposition 5. *Let $\langle \rho, \lambda \rangle$ be a balanced and persistent resampling scheme. If $\langle \rho, \lambda \rangle$ is reindexable then invariance still holds if Line 1 of Algorithm 2 is replaced by*

1': Sample $(K, U, \mathbf{A}) \sim \lambda'(k|a)\rho^k(u|a)\delta_a(a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k, u)$,

where $\lambda'(\,\cdot\,|n)$ is an arbitrary distribution on $[M]$, for any $n \in [N]$.

The reindexability assumption of Proposition 5 is satisfied for exchangeable resampling schemes.

Definition 11 (exchangeability). *A resampling scheme $\langle \rho, \lambda \rangle$ is* exchangeable *if the marginal distribution $\rho(\mathbf{a})$ is exchangeable.*

Proposition 6. *A resampling scheme is reindexable if it is exchangeable.*

Remark 2. *For non-exchangeable resampling schemes, reindexability can be satisfied by permuting or shifting the order of the ancestor indices uniformly at random. Our work shows that such random permutations/shifts are only necessary if one wishes to implement conditional resampling in such a way that K is always set to 1 or M, say, in Line 1 of Algorithm 2. Otherwise, they only add computational cost. More details are given in Appendix B.1.*

3 Examples

We now demonstrate that a number of well known resampling schemes $\langle \rho, \lambda \rangle$ can be viewed as special cases of our framework. In particular, this allows us to present valid *conditional* versions of popular resampling schemes that are not exchangeable, without the need for randomly permuting or shifting the order of the generated ancestor indices.

We start with a number of *elementary* resampling schemes (multinomial, stratified, systematic or killing resampling and even 'trivial' resampling, i.e., not resampling). Then, we discuss *composite* resampling schemes which build more sophisticated resampling schemes from elementary ones.

Remark 3. *For balanced resampling schemes, the auxiliary variables U can be integrated out without changing the law of $(\widetilde{X}^{1:M}, \widetilde{w}^{1:M})$ conditional on $\mathcal{F}$. Hence, we sometimes include U in the balanced resampling schemes below purely to simplify the presentation. But we could equally work with $\mathsf{U} = \emptyset$.*

3.1 Elementary resampling schemes

The popular elementary resampling schemes discussed in this section are persistent and balanced and all (except 'trivial' resampling) are even weight-homogeneous.

3.1.1 Multinomial resampling

Multinomial resampling (Rubin, 1987)—also known as *stochastic sampling with replacement* in the genetic algorithms literature (Baker, 1987)—takes $\mathsf{U} := \emptyset$ and $\rho(\mathbf{a}) := \prod_{m=1}^{M} \rho^m(a^m)$, where $\rho^m := \mathrm{Cat}(\mathbf{W})$, and $\lambda(m|n) := \mathrm{Unif}_{[M]}(m)$ and hence $\phi \equiv 1$. Clearly, multinomial resampling is weight-homogeneous (and hence balanced and persistent) and also exchangeable (so that Proposition 5 would allow us to simply set $K := 1$, say, in Algorithm 4). Line 2 of Algorithm 3 and Line 2 of Algorithm 4 can be implemented in $\mathcal{O}(\max\{M, N\})$ operations using *aliasing* methods (Walker, 1974, 1977).

Algorithm 3 multinomial resampling

1: Sample $A^m \sim \mathrm{Cat}(\mathbf{W})$, for $m \in [M]$.
2: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := 1/M$, for $m \in [M]$.

Algorithm 4 conditional multinomial resampling

Require: $a \in [N]$ with $W^a > 0$.
1: Sample $K \sim \mathrm{Unif}_{[N]}$ and set $A^K := a$.
2: Sample $A^m \sim \mathrm{Cat}(\mathbf{W})$, for $m \in [M] \setminus \{K\}$.
3: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := 1/M$, for $m \in [M]$.

3.1.2 Stratified resampling

Stratified resampling (Kitagawa, 1996) takes $\mathsf{U} \coloneqq [0,1]^M$ and, writing $U = U^{1:M}$:

$$\rho(\mathbf{a}, \mathrm{d}u) \coloneqq \prod_{m=1}^{M} \mathrm{Unif}_{[0,1]}(\mathrm{d}u^m)\delta_{F^{-1}([u^m+m-1]/M)}(a^m),$$

where $F(n) \coloneqq \sum_{l=1}^{n} W^l$, for $n \in [N]$ and its generalized invers $F^{-1}(p) \coloneqq \min\{n \in [N] \mid p \leq F(n)\}$, for $p \in [0,1]$. For what follows, it will be convenient to use that for any $u \in [0,1]$:

$$n = F^{-1}\left(\frac{u+m-1}{M}\right) \iff u \in I^m(n),$$

for any $(m,n,u) \in [M] \times [N] \times [0,1]$, where $\{I^m(n)\}_{n\in[N]}$ is the partition of $(0,1]$ given by

$$I^m(n) \coloneqq (\max\{0, MF(n-1) - m + 1\}, \min\{1, MF(n) - m + 1\}].$$

with convention $F(0) \coloneqq 0$. With this notation, we can express the resampling distribution equivalently as $\rho(\mathbf{a}, \mathrm{d}u) \coloneqq \prod_{m=1}^{M} \rho^m(a^m)\,\mathrm{Unif}_{I^m(a^m)}(\mathrm{d}u^m)$, where

$$\rho^m(n) = \mathrm{Cat}(n; (\mathrm{vol}(I^m(l)))_{l\in[N]}). \tag{6}$$

Stratified resampling is not exchangeable. However, ρ satisfies the conventional 'unbiasedness' condition (5):

$$\sum_{m=1}^{M} \rho^m(n) = \sum_{m=1}^{M} \mathrm{vol}((\max\{m-1, MF(n-1)\}, \min\{m, MF(n)\}]) = MW^n.$$

Consequently, stratified resampling is weight-homogeneous (and hence balanced and thus properly weighted as well as persistent) with

$$\begin{aligned}\lambda(m,u|n) &\coloneqq \frac{\rho^m(n,u)}{MW^n} \\ &= \mathrm{Cat}(m; (\mathrm{vol}(I^l(n))/(MW^n))_{l\in[M]})\,\mathrm{Unif}_{I^m(n)}(u^m)\prod_{\substack{l=1\\l\neq m}}^{M}\mathrm{Unif}_{[0,1]}(u^l).\end{aligned}$$

Algorithm 5 stratified resampling

1: Sample $U^m \sim \mathrm{Unif}_{[0,1]}$, for $m \in [M]$.
2: Set $A^m \coloneqq F^{-1}([U^m + m - 1]/M)$, for $m \in [M]$, via Algorithm 7.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

Note that in Algorithm 6, by construction, $A^K = a$ with probability 1. It is well known that Line 2 in Algorithm 5 and Line 3 in Algorithm 6 can be implemented in $\mathcal{O}(\max\{M,N\})$ operations as outlined in Algorithm 7.

Algorithm 6 conditional stratified resampling

Require: $a \in [N]$ with $W^a > 0$.

1: Sample $K \sim \mathrm{Cat}((\mathrm{vol}(I^m(a))/(MW^a))_{m\in[M]})$.
2: Sample $U^K \sim \mathrm{Unif}_{I^K(a)}$, and $U^m \sim \mathrm{Unif}_{[0,1]}$ for $m \in [M] \setminus \{K\}$.
3: Set $A^m := F^{-1}([U^m + m - 1]/M)$, for $m \in [M]$, via Algorithm 7.
4: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := 1/M$, for $m \in [M]$.

Algorithm 7 efficient stratified inverse transform method

Require: $W^{1:N} \in [0,1]^N$ with $\sum_{n=1}^N W^n = 1$ and $U^{1:M} \in [0,1]^M$.

1: Set $Q^n := \sum_{l=1}^n W^l$, for $n \in [N]$, and $T^m := (U^m + m - 1)/M$, for $m \in [M]$.
2: Set $i := 1$, $j := 1$.
3: **while** $j \leq M$ **do**
4: **if** $T^j \leq Q^j$ **then** set $A^j := i$ and $j \leftarrow j + 1$.
5: **else** Set $i \leftarrow i + 1$.
6: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := 1/M$, for $m \in [M]$.

3.1.3 Systematic resampling

Systematic resampling (Carpenter et al., 1999)—also known as *stochastic universal sampling* in genetic algorithms (Baker, 1987)—differs from stratified resampling in that it shares the same uniform random variable U across all strata:

$$\rho(\mathbf{a}, u) := \mathrm{Unif}_{[0,1]}(u) \prod_{m=1}^{M} \delta_{F^{-1}([u+m-1]/M)}(a^m),$$

with $\mathsf{U} := [0,1]$. Thus, $\rho^m(n) = \mathrm{vol}(I^m(n))$ as in stratified resampling (6) so that systematic resampling again satisfies the conventional 'unbiasedness' condition (5); and setting

$$\lambda(m, u|n) := \frac{\rho^m(n,u)}{MW^n} = \mathrm{Cat}(m; (\mathrm{vol}(I^l(n))/(MW^n))_{l\in[M]})\,\mathrm{Unif}_{I^m(n)}(u),$$

ensures that systematic resampling is weight-homogeneous (and hence balanced and thus properly weighted as well as persistent).

Algorithm 8 systematic resampling

1: Sample $U \sim \mathrm{Unif}_{[0,1]}$.u
2: Set $A^m := F^{-1}([U + m - 1]/M)$, for $m \in [M]$ via Alg. 7 with $U^1 = \ldots = U^M = U$.
3: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := 1/M$, for $m \in [M]$.

Previously, Chopin and Singh (2015, Algorithm 4) derived a version of conditional systematic resampling which coincides with Algorithm 9 up to an additional random shift (see Remark 2).

Algorithm 9 conditional systematic resampling

Require: $a \in [N]$ with $W^a > 0$.

1: Sample $K \sim \mathrm{Cat}((\mathrm{vol}(I^m(a))/(MW^a))_{m\in[M]})$ and $U \sim \mathrm{Unif}_{I^K(a)}$.
2: Set $A^m \coloneqq F^{-1}([U+m-1]/M)$, for $m \in [M]$ via Alg. 7 with $U^1 = \ldots = U^M = U$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

3.1.4 Killing resampling

Let $M = N$. *Killing* resampling (Chopin et al., 2022) (see also Del Moral, 2004, Section 2.5.3, for a closely related scheme) takes $\mathsf{U} \coloneqq \emptyset$, $\rho(\mathbf{a}) \coloneqq \prod_{m=1}^M \rho^m(a^m)$, where, writing $W^\star \coloneqq \max\{W^1, \ldots, W^N\}$,

$$\rho^m(n) \coloneqq \frac{W^m}{W^\star}\delta_m(n) + \left(1 - \frac{W^m}{W^\star}\right)\mathrm{Cat}(n; \mathbf{W}).$$

It can be verified that ρ thus satisfies the conventional 'unbiasedness' condition (5). Thus, taking

$$\lambda(m|n) \coloneqq \frac{\rho^m(n)}{MW^n} = \frac{W^\star - W^n + 1}{MW^\star}\mathbf{1}\{m = n\} + \frac{W^\star - W^m}{MW^\star}\mathbf{1}\{m \in [M] \setminus \{n\}\}, \quad (7)$$

means that killing resampling is weight-homogeneous (and hence properly weighted and persistent).

Algorithm 10 killing resampling

1: **for** $m \in [M]$ **do**
2: **with probability** $W^m/W^\star$ set $A^m \coloneqq m$.
3: **otherwise** sample $A^m \sim \mathrm{Cat}(\mathbf{W})$.
4: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

Algorithm 11 conditional killing resampling

Require: $a \in [N]$ with $W^a > 0$.

1: Sample $K \sim \lambda(\cdot\,|a)$ defined in (7) and set $A^K \coloneqq a$.
2: **for** $m \in [M] \setminus \{K\}$ **do**
3: **with probability** $W^m/W^\star$ set $A^m \coloneqq m$.
4: **otherwise** sample $A^m \sim \mathrm{Cat}(\mathbf{W})$.
5: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

Previously, Karppinen et al. (2023) derived a version of conditional killing resampling which coincides with Algorithm 11 up to an additional random shift (see Remark 2).

3.1.5 Trivial resampling (i.e., not resampling)

We now show that 'not resampling' can be viewed as a resampling scheme (termed *trivial* resampling) within our framework. This will become useful in Section 3.2.3 where justify adaptive resampling as mixing trivial resampling with another, 'non-trivial' resampling scheme.

Let $M \coloneqq N$. Then trivial resampling can be formalised as $\mathsf{U} \coloneqq \emptyset$, $\rho(\mathbf{a}) \coloneqq \prod_{m=1}^{M} \rho^m(a^m)$, where $\rho^m \coloneqq \delta_m$. With $\lambda(\cdot\,|n) \coloneqq \delta_n$, trivial resampling is then balanced (and hence properly weighted) and persistent, and has $\phi(m, n, u) = MW^n \mathbf{1}\{m = n\}$.

Algorithm 12 trivial resampling

1: Set $A^m \coloneqq m$, for $m \in [M]$.
2: Set $\widetilde{X}^m \coloneqq X^m$ and $\widetilde{w}^m \coloneqq W^m$, for $m \in [M]$.

Algorithm 13 conditional trivial resampling

Require: $a \in [N]$ with $W^a > 0$.
1: Set $K \coloneqq a$ and $A^K \coloneqq a$.
2: Set $A^m \coloneqq m$, for $m \in [M] \setminus \{K\}$.
3: Set $\widetilde{X}^m \coloneqq X^m$ and $\widetilde{w}^m \coloneqq W^m$, for $m \in [M]$.

3.2 Composite resampling schemes

Let $\langle \varrho, \kappa \rangle$ denote some properly weighted and persistent resampling scheme (e.g., one of the elementary resampling schemes from the previous section). In this section, we illustrate that $\langle \varrho, \kappa \rangle$ can be modified to obtain a new properly weighted and persistent resampling scheme $\langle \rho, \lambda \rangle$ with potentially more favourable properties. This permits a simple justification of many sophisticated resampling schemes that have been proposed in the literature such as residual resampling, resampling with weight re-ordering, adaptive resampling, optimal finite-state resampling and chopthin resampling. In particular, our framework shows how to implement conditional resampling for each of these.

3.2.1 Residual resampling

Let $M_0 \coloneqq \sum_{n=1}^{N} \lfloor MW^n \rfloor$ and

$$\alpha = \alpha^{1:M_0} = (\overbrace{1, \ldots, 1}^{\lfloor MW^1 \rfloor \text{ times}}, \overbrace{2, \ldots, 2}^{\lfloor MW^2 \rfloor \text{ times}}, \ldots, \overbrace{N, \ldots, N}^{\lfloor MW^N \rfloor \text{ times}}).$$

Additionally, let $\langle \varrho, \kappa \rangle$ be some weight-homogeneous resampling scheme (e.g. any of the elementary resampling schemes from Section 3.1 except for 'trivial' resampling) which generates $M - M_0$ ancestor indices on $[N]$ based on weights $\mathbf{V} \coloneqq V^{1:N}$, where

$$V^n \coloneqq \frac{MW^n - \lfloor MW^n \rfloor}{M - M_0}.$$

Residual resampling then takes the following resampling distribution (other orderings of $\mathbf{a} = a^{1:M}$ are possible and can facilitate 'in place' implementations):

$$\rho(\mathbf{a}, u) \coloneqq \delta_\alpha(a^{1:M_0}) \sum_{\mathbf{b} \in [N]^{M-M_0}} \varrho(\mathbf{b}, u; \mathbf{V}) \delta_{\mathbf{b}}(a^{(M_0+1):M}).$$

Consequently, $\rho^m(n) = \mathbf{1}\{m \in [M_0]\}\delta_{\alpha^m}(n) + \mathbf{1}\{m \in [M] \setminus [M_0]\}\varrho^{m-M_0}(n; \mathbf{V})$. It can be verified that ρ satisfies the conventional 'unbiasedness' condition (5). Thus, setting

$$\lambda(m, u|n) = \frac{\rho^m(n, u)}{MW^n} = p^n \operatorname{Unif}_{\{l \in [M_0] | \alpha^l = n\}}(m)\varrho(u) + (1 - p^n)\kappa(m - M_0, u|n; \mathbf{V}),$$

where $p^n \coloneqq \lfloor MW^n \rfloor / (MW^n)$, makes the residual resampling scheme $\langle \rho, \lambda \rangle$ is weight-homogenous.

If $\langle \varrho, \kappa \rangle$ is multinomial resampling then $\langle \rho, \lambda \rangle$ is called *residual-multinomial resampling* (Liu and Chen, 1998)—also called *remainder stochastic sampling with replacement* in genetic algorithms (Baker, 1987)—but other choices, e.g., *residual-systematic resampling* (Higuchi, 1995) are possible.

Algorithm 14 residual resampling

1: Sample $(\mathbf{B}, U) \coloneqq (B^{1:(M-M_0)}, U) \sim \varrho(\,\cdot\,; \mathbf{V})$.
2: Set $\mathbf{A} \coloneqq (\alpha, \mathbf{B})$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

Algorithm 15 conditional residual resampling

Require: $a \in [N]$ with $W^a > 0$.
1: Sample $V \sim \operatorname{Unif}_{[0,1]}$.
2: **if** $V \leq \lfloor MW^a \rfloor / (MW^a)$ **then**
3: Sample $K \sim \operatorname{Unif}_{\{m \in [M_0] | \alpha^m = a\}}$.
4: Sample $(\mathbf{A}, U)$ via Algorithm 14.
5: **else**
6: Sample $(L, \mathbf{B}, U) \sim \kappa(l, u|a; \mathbf{V})\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l, u; \mathbf{V})$.
7: $K \coloneqq M_0 + L$ and $\mathbf{A} \coloneqq (\alpha, \mathbf{B})$.
8: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq 1/M$, for $m \in [M]$.

Previously, Chopin and Singh (2015, Algorithm 2) derived a version of conditional residual-multinomial resampling which coincides with Algorithm 15 up to an additional random shift (see Remark 2).

3.2.2 Re-ordering weights prior to resampling

Let $\langle \varrho, \kappa \rangle$ be a balanced and persistent resampling scheme. If $\langle \varrho, \kappa \rangle$ is non-exchangeable, then it can sometimes be desirable to re-order the weights before resampling, e.g., to improve convergence rates (Kitagawa, 1996; Douc et al., 2005; Gerber et al., 2019).

More formally, let $\varpi \in \mathsf{S}_N$ be some $\mathcal{F}$-measurable permutation (we only consider deterministic weight reordering for simplicity but random reorderings would be possible, too). Furthermore, write $\mathbf{W}^{\varpi} = (W^{\varpi(1)}, \dots, W^{\varpi(N)})$. We also now include the (order of the) weights explicitly into the resampling distribution. Then, sampling $(\mathbf{B}, U) = (B^{1:M}, U) \sim \varrho(\,\cdot\,; \mathbf{W}^{\varpi})$—and subsequently replacing the mth generated ancestor index B^m by $A^m \coloneqq \varpi(B^m)$—induces another resampling scheme $\langle \rho, \lambda \rangle$ with

$$\rho(\mathbf{a}, u) \coloneqq \sum_{\mathbf{b} \in [N]^M} \varrho(\mathbf{b}, u; \mathbf{W}^{\varpi}) \delta_{\varpi(\mathbf{b})}(\mathbf{a}) = \varrho(\varpi^{-1}(\mathbf{a}), u; \mathbf{W}^{\varpi}),$$
$$\lambda(m, u|n) \coloneqq \frac{\varrho^m(\varpi^{-1}(n); \mathbf{W}^{\varpi})}{\sum_{l=1}^M \varrho^l(\varpi^{-1}(n); \mathbf{W}^{\varpi})} \varrho^m(u|\varpi^{-1}(n); \mathbf{W}^{\varpi}) = \kappa(m, u|\varpi^{-1}(n), \mathbf{W}^{\varpi}),$$

where, for any $\mathbf{b} \coloneqq b^{1:N} \in [N]^N$ and any permutation $\sigma \in \mathsf{S}_N$, we write $\sigma(\mathbf{b}) \coloneqq (\sigma(b^1), \dots, \sigma(b^N))$. By assumption on $\langle \varrho, \kappa \rangle$, $\langle \rho, \lambda \rangle$ is then balanced (and hence properly weighted) and persistent.

Algorithm 16 resampling with weight re-ordering

Require: a permutation $\varpi \in \mathsf{S}_N$ which may depend on $(\mathbf{X}, \mathbf{W})$.
1: Sample $(\mathbf{B}, U) = (B^{1:M}, U) \sim \varrho(\,\cdot\,; \mathbf{W}^{\varpi})$.
2: Set $\mathbf{A} \coloneqq \varpi(\mathbf{B})$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^M \varrho^l(\varpi^{-1}(A^m); \mathbf{W}^{\varpi})$, for $m \in [M]$.

Algorithm 17 conditional resampling with weight re-ordering

Require: $a \in [N]$ with $W^a > 0$; a permutation $\varpi \in \mathsf{S}_N$ which may depend on $(\mathbf{X}, \mathbf{W})$.
1: Sample $(K, \mathbf{B}, U) \sim \kappa(k, u|\varpi^{-1}(a); \mathbf{W}^{\varpi}) \delta_{\varpi^{-1}(a)}(b^k) \varrho^{-k}(\mathbf{b}^{-k}|b^k, u; \mathbf{W}^{\varpi})$.
2: Set $\mathbf{A} \coloneqq \varpi(\mathbf{B})$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^M \varrho^l(\varpi^{-1}(A^m); \mathbf{W}^{\varpi})$, for $m \in [M]$.

In particular, note that if $\langle \varrho, \kappa \rangle$ is weight-homogeneous (which holds for all the elementary resampling schemes discussed in Section 3.1 except for 'trivial' resampling) then $\langle \rho, \lambda \rangle$ is again weight-homogeneous. If $\langle \varrho, \kappa \rangle$ is the stratified or systematic resampling scheme and if ϖ is a mean partition order for $\mathbf{W}$, i.e., $W^{\varpi(1)}, \dots, W^{\varpi(N_0)} \leq \frac{1}{N} \sum_{n=1}^N W^n$, and $W^{\varpi(N_0)}, \dots, W^{\varpi(N)} > \frac{1}{N} \sum_{n=1}^N W^n$, for some $N_0 \in [N]$, then Algorithm 16 recovers the mean-order versions of stratified/systematic resampling proposed in Chopin et al. (2022); Karppinen et al. (2023) recently justified its conditional version by using random shifts.

3.2.3 Mixture resampling and adaptive resampling

For some $I \in \mathbb{N}$, let $\{\langle \varrho_i, \kappa_i \rangle\}_{i \in [I]}$ be family of balanced and persistent resampling schemes each of which may make use of some auxiliary variables U_i taking values in some space U_i. Additionally, let $V \in [I]$ be some random indicator whose distribution $\rho(v) = \rho(v; \mathbf{W})$

may depend on the weights $\mathbf{W} = W^{1:N}$ although we do not make this dependence explicit in the notation.

We will now specify a *mixture resampling* scheme $\langle \rho, \lambda \rangle$ which performs resampling according to $\langle \varrho_v, \kappa_v \rangle$ with probability $\rho(v)$: Write $\mathsf{U} \coloneqq [I] \times \bigtimes_{i=1}^{I} \mathsf{U}_i$ as well as $U \coloneqq (V, U_{1:I})$ and define

$$\rho(\mathbf{a}, u) \coloneqq \rho(v)\varrho_v(\mathbf{a}, u_v)\prod_{\substack{i=1 \\ i \neq v}}^{I} \varrho_i(u_i),$$

and (whenever $W^n > 0$):

$$\lambda(m, u|n) \coloneqq \frac{\rho(v)\varrho_v^m(n, u_v)}{\sum_{l=1}^{M} \varrho_v^l(n)} \prod_{\substack{i=1 \\ i \neq v}}^{I} \varrho_i(u_i) = \rho(v)\kappa_v(m, u_v|n) \prod_{\substack{i=1 \\ i \neq v}}^{I} \varrho_i(u_i).$$

Then $\langle \rho, \lambda \rangle$ is properly weighted and persistent but not generally balanced (unless $\rho(i) = 1$ for some $i \in [I]$). To see this, note that $\phi(m, n, u) = MW^n / \sum_{l=1}^{M} \varrho_v^l(n)$ still depends on the component v of u. We could have made $\langle \rho, \lambda \rangle$ balanced by setting $\lambda(m, \mathrm{d}u|n) \propto \rho^m(n, \mathrm{d}u)$. However, then $\phi(m, n, u) = MW^n / \sum_{i=1}^{I} \rho(i) \sum_{l=1}^{M} \varrho_i^l(n)$, i.e., the cost of evaluating post-resampling weights grows with the number of mixture components, I.

Algorithm 18 mixture resampling

1: Sample $V \sim \rho(v)$ and $(\mathbf{A}, U_V) \sim \varrho_V(\mathbf{a}, u_V)$;
2: Set $\widetilde{X}^m \coloneqq X^{A^m}$ as well as $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^{M} \varrho_V^l(A^m)$, for $m \in [M]$.

Algorithm 19 conditional mixture resampling

Require: $a \in [N]$ with $W^a > 0$.
1: Sample $V \sim \rho(v)$.
2: Sample $(K, \mathbf{A}, U_V) \sim \kappa_V(k, u_V|a)\delta_a(a^k)\varrho_V^{-k}(\mathbf{a}^{-k}|a^k, u_V)$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ as well as $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^{M} \varrho_V^l(A^m)$, for $m \in [M]$.

A special case of such a mixture resampling scheme is frequently employed within *SMC* methods (to be discussed in Section 4). Assume that $M = N$, $I = 2$, and that $\langle \varrho_2, \kappa_2 \rangle$ is the 'trivial' resampling scheme from Section 3.1.5. A common choice, first proposed in Liu and Chen (1995), is then to decide 'whether or not to resample' based on some $\mathcal{F}$-measurable function such as the *effective sample size (ESS)* of the weights (Kong et al., 1994):

$$\mathrm{ess}(\mathbf{W}) \coloneqq \left[\sum_{n=1}^{N} (W^n)^2\right]^{-1},$$

which corresponds to the degenerate distribution

$$\rho(v) = \rho(v; \mathbf{W}) \coloneqq \mathbf{1}\{\mathrm{ess}(\mathbf{W}) \leq \eta N\}\delta_1(v) + \mathbf{1}\{\mathrm{ess}(\mathbf{W}) > \eta N\}\delta_2(v),$$

for some user-defined threshold $\eta \in [0, 1]$. In the remainder of this work, we refer to this special case of mixture resampling as *ESS-adaptive resampling*.

3.2.4 Partitioned resampling, partial resampling and optimal finite-state resampling

Let $I \in \mathbb{N}$ and for $i \in [I]$, let $L_i \coloneqq \{l_i^1, \dots, l_i^{N_i}\} \subseteq [N]$, be such that $\{L_1, \dots, L_I\}$ is a partition of $[N]$. Then we can 'partition' the weighted sample $(\mathbf{X}, \mathbf{W})$ into I disjoint blocks of weighted samples of the form $(\mathbf{X}_i, \mathbf{W}_i) \coloneqq (X_i^{1:N_i}, W_i^{1:N_i})$, where for all $n \in [N_i]$:

$$X_i^n \coloneqq X^{l_i^n}, \qquad \text{and} \qquad W_i^n \coloneqq \frac{W^{l_i^n}}{\sum_{m=1}^{N_i} W^{l_i^m}}.$$

For $i \in [I]$, let $\langle \varrho_i, \kappa_i \rangle$ be some balanced and persistent resampling scheme which generates M_i ancestor indices $\mathbf{A}_i = A_i^{1:M_i} \in [N_i]^{M_i}$ for the weighted sample $(\mathbf{X}_i, \mathbf{W}_i)$ and may make use of some auxiliary variables U_i taking values in some space U_i.

We will now specify a *partitioned resampling* scheme $\langle \rho, \lambda \rangle$ which generates ancestor indices $\mathbf{A} \in \bigtimes_{i=1}^I L_i^{M_i}$ from $\mathbf{A}_1, \dots, \mathbf{A}_I$ by setting

$$\mathbf{A} \coloneqq h(\mathbf{A}_1, \dots, \mathbf{A}_I) \coloneqq (l_1^{A_1^1}, \dots, l_1^{A_1^{M_1}}, l_2^{A_2^1}, \dots, l_2^{A_2^{M_2}}, \dots, l_I^{A_I^1}, \dots, l_I^{A_I^{M_I}}).$$

More precisely, we set $M \coloneqq \sum_{i=1}^I M_i$, $\mathsf{U} \coloneqq \bigtimes_{i=1}^I \mathsf{U}_i$, $U \coloneqq U_{1:I}$ and

$$\begin{aligned}
\rho(\mathbf{a}, u) &\coloneqq \sum_{(\mathbf{a}_1, \dots, \mathbf{a}_I) \in \bigtimes_{i=1}^I [N_i]^{M_i}} \delta_{h(\mathbf{a}_1, \dots, \mathbf{a}_I)}(\mathbf{a}) \prod_{i=1}^I \varrho_i(\mathbf{a}_i, u_i), \\
\lambda(m, u | n) &\coloneqq \frac{\rho^m(n, u)}{\sum_{m=1}^M \rho^m(n)} = \kappa_{i(n)}\Big(m - \sum_{j=1}^{i(n)-1} M_j, u_{i(n)} \,\Big|\, q(n)\Big) \prod_{\substack{j=1 \\ j \neq i(n)}}^I \varrho_j(u_j), \\
\phi(m, n, u) &= \frac{W^n \lambda(m, u | n)}{M^{-1} \rho^m(n, u)} = \frac{M W^n}{\sum_{l=1}^{M_{i(n)}} \varrho_{i(n)}^l(q(n))},
\end{aligned}$$

where $i(n) \in [I]$ and $q(n) \in [N_{i(n)}]$ are the unique solutions to $n \in L_{i(n)}$ and $n = l_{i(n)}^{q(n)}$, respectively. Then $\langle \rho, \lambda \rangle$ is again balanced (and thus properly weighted) and persistent.

Algorithm 20 partitioned resampling

1: Sample $(\mathbf{A}_i, U_i) \sim \varrho_i(\mathbf{a}_i, u_i; \mathbf{W}_i)$, for $i \in [I]$.
2: Set $\mathbf{A} \coloneqq h(\mathbf{A}_1, \dots, \mathbf{A}_I)$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ as well as $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^{M_{i(A^m)}} \varrho_{i(A^m)}^l(q(A^m))$, for $m \in [M]$.

We end this section with two special cases of partitioned resampling that have been proposed in the literature. Let $I = 2$ and let $\langle \varrho_1, \kappa_1 \rangle$ be the trivial resampling scheme from Section 3.1.5 (with $M_1 = N_1$) and let $\langle \varrho_2, \kappa_2 \rangle$ be some other weight-homogeneous (for simplicity) resampling scheme.

Then $\langle \rho, \lambda \rangle$ reduces to the *partial resampling* scheme from Martino et al. (2016) (a related method was proposed in Bolić et al. 2004) which resamples only the particles with indices in L_2 but keeps those with indices in L_1 unchanged. Weight-homogeneity

Algorithm 21 conditional partitioned resampling

Require: $a \in [N]$ with $W^a > 0$.

1: Sample

$$(L, \mathbf{A}_{i(a)}, U_{i(a)}) \sim \kappa_{i(a)}(l, u_{i(a)}|q(a); \mathbf{W}_{i(a)})\delta_{q(a)}(a^l_{i(a)})\varrho^{-l}_{i(a)}(\mathbf{a}^{-l}_{i(a)}|a^l_{i(a)}, u_{i(a)}; \mathbf{W}_{i(a)})$$

as well as $(\mathbf{A}_j, U_j) \sim \varrho_j(\mathbf{a}_j, u_j; \mathbf{W}_j)$, for all $j \in [I] \setminus \{i(a)\}$.

2: Set $\mathbf{A} \coloneqq h(\mathbf{A}_1, \ldots, \mathbf{A}_I)$ and $K \coloneqq L + \sum_{j=1}^{i(a)-1} M_j$.

3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ as well as $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{l=1}^{M_{i(A^m)}} \varrho^l_{i(A^m)}(q(A^m))$, for $m \in [M]$.

of $\langle \varrho_2, \kappa_2 \rangle$ then implies $\sum_{m=1}^{M_2} \varrho_2^m(n) = M_2 W_2^n$, for any $n \in [N_2]$, so that

$$\phi(m, n, u) = \frac{MW^n}{\sum_{l=1}^{M_{i(n)}} \varrho^l_{i(n)}(q(n))} = \begin{cases} MW^n, & \text{if } i(n) = 1, \\ \dfrac{MW^n}{M_2 W_2^{q(n)}} = \dfrac{M}{M_2} \displaystyle\sum_{j=1}^{N_2} W^{l_2^j}, & \text{if } i(n) = 2. \end{cases} \tag{8}$$

A sophisticated version of such a partial resampling scheme known as *optimal finite-state resampling* was proposed in Fearnhead (1998); Fearnhead and Clifford (2003); and its conditional version in Whiteley et al. (2010). It uses Fearnhead (1998, Algorithm 5.2) to solve

$$\sum_{n=1}^{N} \min\{1, CW^n\} = M, \tag{9}$$

for $C > 0$ and then sets $L_1 \coloneqq \{n \in [N] \mid CW^n \geq 1\}$ and $L_2 \coloneqq \{n \in [N] \mid CW^n < 1\}$. By (9), we then have $\frac{1}{M_2} \sum_{n=1}^{N_2} W^{l_2^n} = 1/C$ so that (8) takes the following simple form: $\phi(m, n, u) = MW^n / \min\{1, CW^n\}$. If $M \leq N$, and if $\langle \varrho_2, \kappa_2 \rangle$ is stratified or systematic resampling, this scheme ensures that the family of ancestor indices $\mathbf{A}$ contains no duplicates. This minimises the variance induced by the resampling operation when X is finite which is why optimal finite-state resampling is at the heart of the *discrete particle filters* from Fearnhead (1998); Fearnhead and Clifford (2003) and related SMC and CSMC algorithms for change-point models (Fearnhead and Liu, 2007; Whiteley et al., 2009).

3.2.5 Modified-weight and chopthin resampling

Consider a weight-homogeneous resampling scheme which generates M ancestor indices using weights $W^{1:N}$. Then by the conventional 'unbiasedness' condition (5), the expected number of descendants of the nth particle is MW^n. More generally, we may wish to enforce that the expected number of descendants of the nth particle is $H^n \neq MW^n$, for some $H^1, \ldots, H^N \geq 0$ such that

$$\forall\, n \in [N] : H^n > 0 \text{ if and only if } W^n > 0; \qquad \text{and} \qquad \sum_{n=1}^{N} H^n = M. \tag{10}$$

This can be achieved as follows. Let $\langle \varrho, \kappa \rangle$ be some weight-homogeneous resampling scheme which generates M ancestors based on the weights $V^{1:N}$, where $V^n = H^n/M$. Then, taking

$$\rho(\mathbf{a}, u) \coloneqq \varrho(\mathbf{a}, u; V^{1:N}), \qquad \lambda(m, u|n) \coloneqq \kappa(m, u|n; V^{1:N}),$$

i.e., $\phi(m, n, u) = MW^n/H^n$, gives a generally no longer weight-homogeneous but still balanced and persistent resampling scheme $\langle \rho, \lambda \rangle$ which satisfies (10). Resampling according to alternative weights has a long history (e.g., Pitt and Shephard (1999); Liu (2001); Chen (2010)).

Algorithm 22 modified-weight resampling

Require: $H^{1:N}$ satisfying (10).

1: Sample $(\mathbf{A}, U)$ via some weight-homogeneous resampling scheme (e.g., via Algorithms 3, 5, 8 or 14) but using weights $V^n \coloneqq H^n/M$ instead of W^n, for $n \in [N]$.
2: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq W^{A^m}/H^{A^m}$, for $m \in [M]$.

Algorithm 23 conditional modified-weight resampling

Require: $a \in [N]$ with $W^a > 0$; $H^{1:N}$ satisfying (10).

1: Sample $(K, \mathbf{A}, U)$ via the conditional version of some weight-homogeneous resampling scheme (e.g., via Algorithms 4, 6, 9 or 15) but using weights $V^n \coloneqq H^n/M$ instead of W^n, for $n \in [N]$.
2: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq W^{A^m}/H^{A^m}$, for $m \in [M]$.

As an example of this idea, Gandy and Lau (2015) sought a resampling scheme such that:

1. particles whose weights are below some threshold $\alpha > 0$ have zero or one descendants ('thin');
2. particles whose weights are equal to or above α have one or more offspring ('chop');
3. the variability of the post-resampling 'weights' can be controlled, i.e., for some $\tilde{\beta} \geq 1$ chosen by the user: $\max_{(k,l)\in[M]^2} \widetilde{w}^k/\widetilde{w}^l \leq \tilde{\beta}$.

These goals could be ensured by Algorithm 22 or 23 with systematic resampling as the weight-homogeneous resampling scheme $\langle \varrho, \kappa \rangle$ and $H^n \coloneqq h_{\alpha^*\beta}(W^n)$ defined through the continuous mapping

$$[0, 1] \ni W \mapsto h_{\alpha,\beta}(W) \coloneqq \begin{cases} W/\alpha, & \text{if } W < \alpha, \\ 1, & \text{if } \alpha \leq W < \alpha\beta/2, \\ 2W/(\alpha\beta), & \text{if } \alpha\beta/2 \leq W. \end{cases}$$

Above, given $\beta \geq 4$, $\alpha^* \in (0, 1]$—found via Gandy and Lau (2015, Algorithm 2)—solves

$$\forall\, n \in [N] : h_{\alpha^*\beta}(W^n) > 0 \text{ if and only if } W^n > 0; \qquad \text{and} \qquad \sum_{n=1}^{N} h_{\alpha^*\beta}(W^n) = M \tag{11}$$

Thus, $\widetilde{w}^m = W^{A^m}/H^{A^m}$, i.e., $\widetilde{w}^m = \alpha^*$, if $W^{A^m} < \alpha^*$, and $\widetilde{w}^m = \min\{W^{A^m}, \alpha^*\beta/2\}$, if $W^{A^m} \geq \alpha^*$.

However, Gandy and Lau (2015) instead set $\widetilde{w}^m := W^{A^m} / \sum_{l=1}^{M} \mathbf{1}\{A^l = A^m\}$, if $W^{A^m} \geq \alpha^*$. We now justify this within our framework using an enlarged set of auxiliary variables and an imbalanced index distribution. Write $u := (u_0, \mathbf{b}) \in \mathsf{U}_0 \times [N]^M =: \mathsf{U}$ with $u_0 \in \mathsf{U}_0$ and $\mathbf{b} := b^{1:M} \in [N]^M$ being, respectively, the auxiliary variables and ancestor indices generated by $\varrho(\,\cdot\,; V^{1:N})$ and set:

$$\rho(\mathbf{a}, u) := \varrho(u; V^{1:N})\delta_{\mathbf{b}}(\mathbf{a}),$$

$$\lambda(m, u|n) := \begin{cases} \frac{\rho^m(n,u)}{\sum_{l=1}^{M}\rho^l(n)} = \kappa(m, u_0|n; V^{1:N})\delta_n(b^m)\varrho^{-m}(\mathbf{b}^{-m}|b^m, u_0; V^{1:N}), & \text{if } W^n < \alpha^*, \\ \varrho(u; V^{1:N})\,\mathrm{Unif}_{\{k\in[M]|b^k=n\}}(m), & \text{if } W^n \geq \alpha^*, \end{cases}$$

$$\phi(m, n, u) = \frac{W^n\lambda(m, u|n)}{M^{-1}\rho^m(n, u)} = \begin{cases} M\alpha^*\,\mathbf{1}\{b^m = n\}, & \text{if } W^n < \alpha^*, \\ MW^n\,\mathbf{1}\{b^m = n\}/\sum_{l=1}^{M}\mathbf{1}\{b^l = n\}, & \text{if } W^n \geq \alpha^*. \end{cases}$$

For intuition, note that Particle n is guaranteed to have at least one descendant under ρ whenever $W^n \geq \alpha$; thus, we have greater flexibility in choosing $\lambda(m, u|n)$ whenever $W^n \geq \alpha$.

The resampling scheme $\langle\rho, \lambda\rangle$, summarised in Algorithm 24, is then (the version of) the *chopthin* resampling scheme from Gandy and Lau (2015) which was demonstrated by the authors to outperform systematic resampling in some applications. Having cast chopthin resampling within our framework immediately shows that it is properly weighted and hence unbiased (and also persistent); it also allows us to derive the corresponding conditional version, presented in Algorithm 25, which is novel.

A later version of chopthin resampling (Gandy and Lau, 2016) differs from the one of Gandy and Lau (2015) in that it additionally (a) uses weight reordering with a permutation ϖ such that $W^{\varpi(1)}, \ldots, W^{\varpi(N_0)} < \alpha^*$, and $W^{\varpi(N_0)}, \ldots, W^{\varpi(N)} \geq \alpha^*$, for some $N_0 \in [N]$, and (b) ensures that the post-resampling weights are again normalised, i.e., $\sum_{m=1}^{M}\widetilde{w}^m = 1$. The latter is achieved by adjusting the post-resampling weights as $\widetilde{w}^m = (W^{A^m} + \xi^{A^m})/\sum_{l=1}^{M}\mathbf{1}\{A^l = A^m\}$, if $W^{A^m} \geq \alpha^*$, where ξ^n is a certain random variable satisfying $\mathbb{E}[\xi^n|\mathcal{F}] = 0$ and $\xi^n - W^n \geq 0$, almost surely. Whether (b) can be justified within our framework (e.g., in order to derive a conditional version) remains an open question.

4 Application to sequential Monte Carlo methods

In this section, we apply our resampling framework to *sequential Monte Carlo (SMC)* (Gordon et al., 1993; Del Moral, 2004) and *conditional sequential Monte Carlo (CSMC)*

Algorithm 24 chopthin resampling

1: Find α^* satisfying (11) via Gandy and Lau (2015, Algorithm 2); set $H^n := h_{\alpha^*\beta}(W^n)$, for $n \in [N]$.
2: Sample $(\mathbf{A}, U)$ via systematic resampling (Algorithm 8) but using weights $V^n := H^n/M$ instead of W^n, for $n \in [N]$.
3: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := \mathbf{1}\{W^{A^m} < \alpha^*\}\alpha^* + \mathbf{1}\{W^{A^m} \geq \alpha^*\}W^{A^m} / \sum_{l=1}^M \mathbf{1}\{A^l = A^m\}$, for $m \in [M]$.

Algorithm 25 conditional chopthin resampling

Require: $a \in [N]$ with $W^a > 0$.
1: Find α^* satisfying (11) via Gandy and Lau (2015, Algorithm 2); set $H^n := h_{\alpha^*\beta}(W^n)$, for $n \in [N]$.
2: **if** $W^a < \alpha^*$ **then**
3: Sample $(K, \mathbf{A}, U)$ via conditional systematic resampling (Algorithm 9) but using weights $V^n := H^n/M$ instead of W^n, for $n \in [N]$.
4: **else**
5: Sample $(\mathbf{A}, U)$ via systematic resampling (Algorithm 8) but using weights $V^n := H^n/M$ instead of W^n, for $n \in [N]$.
6: Sample $K \sim \mathrm{Unif}_{\{l \in [M] | A^l = a\}}$.
7: Set $\widetilde{X}^m := X^{A^m}$ and $\widetilde{w}^m := \mathbf{1}\{W^{A^m} < \alpha^*\}\alpha^* + \mathbf{1}\{W^{A^m} \geq \alpha^*\}W^{A^m} / \sum_{l=1}^M \mathbf{1}\{A^l = A^m\}$, for $m \in [M]$.

(Andrieu et al., 2010) methods (see Chopin and Papaspiliopoulos, 2020, for further references). Specifically, our framework gives simple proofs that SMC and CSMC algorithms can be implemented in a valid manner even if the resampling scheme (a) violates the ('unbiasedness', exchangeability or 'marginal unbiasedness') conditions commonly imposed in the literature, e.g., in Andrieu et al. (2010); Chopin and Singh (2015); Karppinen et al. (2023); (b) has a complicated dependence structure, e.g., as in systematic or chopthin resampling; or (c) is adaptive.

4.1 Sequential Monte Carlo

Let $M_t(\cdot|x_{t-1})$ be conditional distributions on X ('mutation kernels') and $G_t \colon \mathsf{X}^2 \to (0, \infty)$ ('potential functions'). Our goal is to estimate expectations w.r.t. the distribution $\pi_T(\mathrm{d}x_{1:T}) := \gamma_T(\mathrm{d}x_{1:T})/\mathcal{Z}_T$, where $\gamma_T(\mathrm{d}x_{1:T}) := \prod_{t=1}^T G_t(x_{t-1:t})M_t(\mathrm{d}x_t|x_{t-1})$, with typically unknown normalising constant $\mathcal{Z}_T := \int_{\mathsf{X}^T} \gamma_T(\mathrm{d}x_{1:T})$; we use the convention that any quantity with subscript $t < 1$ is to be ignored, i.e., $M_1(\mathrm{d}x_1|x_0) = M_1(\mathrm{d}x_1)$ and $G_1(x_{0:1}) = G_1(x_1)$.

The SMC algorithm is outlined in Algorithm 26. It uses $N_t \in \mathbb{N}$ particles at time t. For simplicity, we assume that N_t is deterministic but random numbers of particles could easily be accommodated by further extending the state space to explicitly include

N_t. Additionally, we let λ_1 be some distribution on $[N_1]$ (usually, $\lambda_1 \coloneqq \mathrm{Unif}_{[N_1]}$). For any $t > 1$, we let $\langle \rho_{t-1}, \lambda_t \rangle$ be a properly weighted and persistent resampling scheme which draws N_t ancestors based on N_{t-1} weights $\mathbf{W}_{t-1} = W_{t-1}^{1:N_{t-1}}$ that are defined below and which specifies post-resampling weights via

$$\phi_t(m, n, u; \mathbf{W}_{t-1}) \coloneqq \frac{W_{t-1}^n \lambda_t(m, u | n; \mathbf{W}_{t-1})}{N_t^{-1} \rho_{t-1}^m(n, u; \mathbf{W}_{t-1})}. \tag{12}$$

Algorithm 26 SMC

for $t = 1, \ldots, T$ **do**
  if $t = 1$ **then**
    Set $\widetilde{w}_{t-1}^n \coloneqq \lambda_1(n)$, for $n \in [N_1]$.
  else
    Sample $(\mathbf{A}_{t-1}, U_{t-1}) = (A_{t-1}^{1:N_t}, U_{t-1}) \sim \rho_{t-1}(\,\cdot\,; \mathbf{W}_{t-1})$.
    Set $\widetilde{w}_{t-1}^n \coloneqq \phi_t(n, A_{t-1}^n, U_{t-1}; \mathbf{W}_{t-1})/N_t$ via (12), and $\widetilde{X}_{t-1}^n \coloneqq X_{t-1}^{A_{t-1}^n}$, for $n \in [N_t]$.
  Sample $X_t^n \sim M_t(\,\cdot\,|\widetilde{X}_{t-1}^n)$ for $n \in [N_t]$.
  Set $w_t^n \coloneqq \widetilde{w}_{t-1}^n G_t(\widetilde{X}_{t-1}^n, X_t^n)$, and $W_t^n \coloneqq w_t^n / \sum_{m=1}^{N_t} w_t^m$, for $n \in [N_t]$.

At the end of Algorithm 26, expectations $\pi_T(f_T)$, integrals $\gamma_T(f_T) = \mathcal{Z}_T \pi_T(f)$, for some suitably integrable function $f_T \colon \mathsf{X}^T \to \mathbb{R}$, and the normalising constant $\mathcal{Z}_T$ can be approximated as

$$\hat{\pi}_T(f_T) \coloneqq \sum_{n=1}^{N_T} W_T^n f_T(X_{1:T}^{(n)}), \qquad \hat{\gamma}_T(f_T) \coloneqq \widehat{\mathcal{Z}}_T \hat{\pi}_T(f_T), \qquad \widehat{\mathcal{Z}}_T \coloneqq \prod_{t=1}^{T} \sum_{n=1}^{N_t} w_t^n,$$

where $X_{1:t}^{(n)}$ is recursively defined as $X_1^{(n)} \coloneqq X_1^n$ and $X_{1:t}^{(n)} \coloneqq (X_{1:t-1}^{(A_{t-1}^n)}, X_t^n)$, for $t > 1$.

The following result (proved in Appendix D.3) generalises the unbiasedness of αSMC from Whiteley et al. (2016, Theorem 1(1)) which uses a framework for resampling that encompasses a number of the (adaptive) resampling schemes from Section 3 but not resampling schemes with a complex dependence structure like (adaptive) systematic resampling or chopthin resampling (Appendix B.2 shows that αSMC resampling is a strict special case of our framework).

Proposition 7. *Assume that the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are properly weighted and persistent for any $t \in [T] \setminus \{1\}$ and let $f_T \colon \mathsf{X}^T \to \mathbb{R}$ be some π_T-integrable test function. Then $\mathbb{E}[\hat{\gamma}_T(f_T)] = \gamma_T(f_T)$ and, in particular for $f_T \equiv 1$, $\mathbb{E}[\widehat{\mathcal{Z}}_T] = \mathcal{Z}_T$.*

The following simple example illustrates that proper weighting/unbiasedness of the resampling scheme is not sufficient to ensure convergence of estimators.

Example 7 ('bad' unbiased resampling scheme). *For simplicity, let $N_1 = \ldots = N_T = N$. Consider a resampling scheme $\langle \rho_{t-1}, \lambda_t \rangle$ with $\rho_{t-1}(\mathbf{a}_{t-1}) \coloneqq W_{t-1}^{a_{t-1}^1} \prod_{m=2}^{N} \delta_{a_{t-1}^1}(a_{t-1}^m)$*

(with $\mathsf{U} = \emptyset$). This resampling scheme corresponds to selecting a single ancestor in proportion to its weight and assigning it to all particles in the current generation. If we take $\lambda_t(m|n) \coloneqq \mathrm{Unif}_{[N]}(m)$, then this resampling scheme is also balanced and hence properly weighted as well as persistent. However, it is not the case that $\hat{\gamma}_T(f_T)$ converges in probability to $\gamma_T(f_T)$, as $N \uparrow \infty$, even though this estimate is unbiased.

Take, for example, $T = 2$, $a, b > 0$, $M_1(\mathrm{d}x_1) = \mathrm{N}(\mathrm{d}x_1; 0, a)$, $M_2(\mathrm{d}x_2|x_1) = \mathrm{N}(\mathrm{d}x_2; x_1, b)$, $G_1 \equiv 1$, $G_2 \equiv 1$ and $f_T(x_{1:T}) = x_T$, so that $\gamma_T(f_T) = 0$. Then one can attain via straightforward manipulations that $\hat{\gamma}_T(f_T) \sim \mathrm{N}(0, a + \frac{b}{N})$, for any $N \in \mathbb{N}$. Thus, while $\hat{\gamma}_T(f_T)$ is unbiased, its variance is greater than or equal to a for any $N \in \mathbb{N}$.

4.2 Conditional sequential Monte Carlo

Hereafter, assume that $N_t \geq 2$, for any $t \in [T]$. Then sampling $X'_{1:T}$ conditional on $x_{1:T}$ via the CSMC algorithm from Algorithm 27 induces a Markov kernel P on X^T. The following result is proved in Appendix D.4.

Algorithm 27 CSMC

Require: $x_{1:T} \in \mathsf{X}^T$ with $\pi_T(x_{1:T}) > 0$.

for $t = 1, \ldots, T$ **do**
  if $t = 1$ **then**
    Sample $K_1 \sim \lambda_1$.
    Set $\widetilde{w}^n_{t-1} \coloneqq \lambda_1(n)$, for $n \in [N_1]$.
  else
    Sample $(K_t, \mathbf{A}^{-K_t}_{t-1}, U_{t-1}) \sim \lambda_t(k_t, u_{t-1}|K_{t-1}; \mathbf{W}_{t-1})\rho^{-k_t}_{t-1}(\mathbf{a}^{-k_t}_{t-1}|k_{t-1}, u_{t-1}; \mathbf{W}_{t-1})$.
    Set $A^{K_t}_{t-1} \coloneqq K_{t-1}$.
    Set $\widetilde{w}^n_{t-1} \coloneqq \phi_t(n, A^n_{t-1}, U_{t-1}; \mathbf{W}_{t-1})/N_t$ via (12), and $\widetilde{X}^n_{t-1} \coloneqq X^{A^n_{t-1}}_{t-1}$, for $n \in [N_t]$.
  Set $X^{K_t}_t \coloneqq x_t$, and sample $X^n_t \sim M_t(\cdot\,|\widetilde{X}^n_{t-1})$, for $n \in [N_t] \setminus \{K_t\}$.
  Set $w^n_t \coloneqq \widetilde{w}^n_{t-1} G_t(\widetilde{X}^n_{t-1}, X^n_t)$, and $W^n_t \coloneqq w^n_t / \sum_{m=1}^{N_t} w^m_t$, for $n \in [N_t]$.
Sample $L_T \sim \mathrm{Cat}(\mathbf{W}_T)$ and set $L_t \coloneqq A^{L_{t+1}}_t$, for $t = T-1, \ldots, 1$.
Return $X'_{1:T} \coloneqq (X^{L_1}_1, \ldots, X^{L_T}_T)$.

Proposition 8. *Assume that the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are properly weighted and persistent for any $t \in [T] \setminus \{1\}$. Then P is π_T-invariant.*

Remark 4. *To keep the presentation simple, we have a simple form of CSMC algorithm (sometimes called* CSMC with ancestor tracing*); but Proposition 9 from Appendix D.1 shows that invariance still holds for the* backward sampling *(Whiteley, 2010) and* ancestor sampling *(Lindsten et al., 2014) extensions (Algorithms 32 and 33 in Appendix D.1) which can improve mixing of the induced Markov chain (Lee et al., 2020; Karjalainen et al., 2025).*

Proposition 8 generalises Huggins and Roy (2019, Lemma A.4) which proved the validity of CSMC algorithms based around the αSMC framework and thus encompasses a number of the resampling schemes from Section 3 but not resampling schemes with a complex dependence structure like (adaptive) systematic resampling or chopthin resampling nor the backward or ancestor-sampling extensions.

The following example illustrates that proper weighting/invariance of the (conditional) resampling scheme is not sufficient for ergodicity of the CSMC kernel.

Example 8 ('bad' invariant resampling scheme). *Using the (properly weighted and persistent) resampling scheme from Example 7, let P be the Markov kernel on X^T induced by the CSMC algorithm. Then P is invariant. However, $\rho_{t-1}^{-k_t}(\mathbf{a}_{t-1}^{-k_t}|k_{t-1}) = \prod_{m=1;m\neq k_{t-1}}^{N} \delta_{k_{t-1}}(a_{t-1}^m)$ so that, whenever $X'_{1:T} \sim P(\cdot|x_{1:T})$, $X'_t = x_t$ with probability 1 for any $t < T$. In other words, P cannot be ergodic.*

4.3 A note on (over)simplification

In the case of multinomial resampling (and other exchangeable schemes), the CSMC algorithm remains valid if we fix $K_t \coloneqq 1$, say, instead of sampling K_t from the index distribution, for $t \in [T]$. In the case of multinomial resampling, fixing $K_t \coloneqq 1$, can be implemented straightforwardly using one of the following two strategies whose outcomes are statistically equivalent:

I. Draw N_t ancestor indices via standard multinomial resampling and then overwrite the first ancestor index with 1.

II. Set the first ancestor index to 1 and then draw the remaining ancestor indices via standard multinomial resampling (with $N_t - 1$ instead of N_t descendants).

These strategies are appealing because they allow for existing implementations of 'unconditional' resampling to be used for conditional resampling. One might thus hope that they are valid more generally.

Unfortunately, this is not the case. For instance, as we illustrate in Appendix A, conditional systematic and residual-multinomial resampling implemented using the Strategies I or II can induce a bias in the sense that the CSMC algorithm no longer leaves π_T invariant. Further, these naïve implementations can also worsen the autocorrelation of the Markov chain induced by the CSMC algorithm.

5 Discussion

We have presented a simple framework for guaranteeing the validity of resampling schemes within sequential Monte Carlo (SMC) algorithms and their conditional analogues within conditional sequential Monte Carlo (CSMC) algorithms under minimal assumptions, covering almost all standard resampling mechanisms as well as more exotic approaches such as chopthin resampling, whose conditional version we derive for

the first time. Empirical results demonstrate that neglecting careful conditional resampling can lead to substantial bias. For simplicity, we have focused on basic SMC and CSMC algorithms. However, our arguments straightforwardly extend to models with non-Markovian dynamics, potential functions which depend on the entire history of the state sequence, resampling schemes with a random number of particles, and to the related algorithms from Lee (2011); Del Moral et al. (2015); Finke et al. (2016); Finke and Thiéry (2023); Corenflos and Särkkä (2025); Corenflos and Finke (2024), but the details have been omitted here in the interests of brevity.

Acknowledgments

Large language models were used occasionally to assist with phrasing and with debugging and refining isolated sections of the accompanying code. All original ideas, mathematical derivations, and proofs are entirely the authors' own work, and the manuscript was written predominantly without AI assistance. For the purpose of open access, the authors have applied a Creative Commons Attribution (CC BY) licence to any Author Accepted Manuscript version arising from this submission. Correspondence should be addressed to Axel Finke (axel.finke@ncl.ac.uk).

Funding

This work was supported by EPSRC grant no EP/K032208/1. AMJ acknowledges financial support from the Engineering and Physical Sciences Research Council (EPSRC; grants EP/R034710/1 and EP/T004134/1) and by United Kingdom Research and Innovation (UKRI) via grant EP/Y014650/1, as part of the ERC Synergy project OCEAN. Research of AL was supported by EPSRC grants EP/R034710/1 and EP/Y028783/1.

Data availability

No new data were generated or analysed during this study. The GitHub repository `https://github.com/AxelFinke/resampling-in-conditional-smc-algorithms` provides the code for reproducing the results referenced in Section 4.3 and shown in Appendix A.

References

Andrieu, C., Doucet, A., and Holenstein, R. (2010). Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):269–342. With discussion.

Baker, J. E. (1987). Reducing bias and inefficiency in the selection algorithm. In *Proceedings of the second International Conference on Genetic Algorithms*, volume 206, pages 14–21.

Bolić, M., Djurić, P. M., and Hong, S. (2004). Resampling algorithms for particle filters: A computational complexity perspective. *EURASIP Journal on Advances in Signal Processing*, 5:2267–2277.

Carpenter, J., Clifford, P., and Fearnhead, P. (1999). An improved particle filter for nonlinear problems. *IEE Proceedings – Radar, Sonar and Navigation*, 146(1):2–7.

Chen, R. (2010). Contribution to the discussion on 'Particle Markov chain Monte Carlo methods' by Andrieu, C., Doucet, A., and Holenstein, R. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):306–307.

Chopin, N. and Papaspiliopoulos, O. (2020). *An introduction to sequential Monte Carlo*. Springer.

Chopin, N. and Singh, S. S. (2015). On particle Gibbs sampling. *Bernoulli*, 21(3):1855–1883.

Chopin, N., Singh, S. S., Soto, T., and Vihola, M. (2022). On resampling schemes for particle filters with weakly informative observations. *The Annals of Statistics*, 50(6):3197–3222.

Corenflos, A. and Finke, A. (2024). Particle-MALA and particle-mGRAD: Gradient-based MCMC methods for high-dimensional state-space models. *arXiv preprint arXiv:2401.14868*.

Corenflos, A. and Särkkä, S. (2025). Auxiliary MCMC samplers for parallelisable inference in high-dimensional latent dynamical systems. *Electronic Journal of Statistics*, 19(1):1370–1424.

Corenflos, A., Zhao, Z., Särkkä, S., Sjölund, J., and Schön, T. B. (2024). Conditioning diffusion models by explicit forward-backward bridging. *arXiv preprint arXiv:2405.13794*.

Crisan, D., Del Moral, P., and Lyons, T. (1999). Discrete filtering using branching and interacting particle systems. *Markov Processes and Related Fields*, 5(3):293–318.

Del Moral, P. (2004). *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer.

Del Moral, P. (2013). *Mean Field Simulation for Monte Carlo Integration*. Number 126 in Monographs on Statistics & Applied Probability. Chapman & Hall/CRC.

Del Moral, P., Jasra, A., Lee, A., Yau, C., and Zhang, X. (2015). The alive particle filter and its use in particle Markov chain Monte Carlo. *Stochastic Analysis and Applications*, 33(6):943–974.

Douc, R., Cappé, O., and Moulines, E. (2005). Comparison of resampling schemes for particle filtering. In *Proceedings of the 4th International Symposium on Image and Signal Processing and Analysis*, pages 64–69. IEEE.

Fearnhead, P. (1998). *Sequential Monte Carlo methods in filter theory*. PhD thesis, Department of Statistics, University of Oxford, UK.

Fearnhead, P. and Clifford, P. (2003). On-line inference for hidden Markov models via particle filters. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 65(4):887–899.

Fearnhead, P. and Liu, Z. (2007). On-line inference for multiple changepoint problems. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(4):589–605.

Finke, A. (2015). *On extended state-space constructions for Monte Carlo methods*. PhD thesis, Department of Statistics, University of Warwick, UK.

Finke, A., Doucet, A., and Johansen, A. M. (2016). On embedded hidden Markov models and particle Markov chain Monte Carlo methods. *arXiv preprint arxiv:1610.08962*.

Finke, A. and Thiéry, A. H. (2023). Conditional sequential Monte Carlo in high dimensions. *Annals of Statistics*, 51(2):437–463.

Gandy, A. and Lau, F. D.-H. (2015). The copthin algorithm for resampling. *arXiv preprint arXiv:1502.07532v3*.

Gandy, A. and Lau, F. D.-H. (2016). The chopthin algorithm for resampling. *IEEE Transactions on Signal Processing*, 64(16):4273–4281.

Gerber, M. and Chopin, N. (2015). Sequential quasi Monte Carlo. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 77(3):509–579.

Gerber, M., Chopin, N., and Whiteley, N. (2019). Negative association, ordering and convergence of resampling methods. *The Annals of Statistics*, 47(4):2236–2260.

Gordon, N. J., Salmond, D. J., and Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F, Radar and Signal Processing*, 140(2):107–113.

Higuchi, T. (1995). On the resampling scheme in the filtering procedure of the kitagawa monte carlo filter. Technical Report 556, Institute of Statistical Mathematics Tokyo.

Hu, X.-L., Schön, T. B., and Ljung, L. (2008). A basic convergence result for particle filtering. *IEEE Transactions on Signal Processing*, 56(4):1337–1348.

Huggins, J. H. and Roy, D. M. (2019). Sequential Monte Carlo as approximate sampling: bounds, adaptive resampling via ∞-ESS, and an application to particle Gibbs. *Bernoulli*, 25:584–622.

Jacob, P. E., Lindsten, F., and Schön, T. B. (2019). Smoothing with couplings of conditional particle filters. *Journal of the American Statistical Association*, 115(530):721–729.

Karjalainen, J., Lee, A., Singh, S. S., and Vihola, M. (2025). Mixing time of the conditional backward sampling particle filter. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, page qkaf078.

Karppinen, S., Singh, S. S., and Vihola, M. (2023). Conditional particle filters with bridge backward sampling. *Journal of Computational and Graphical Statistics*, 33(2):364–378.

Kitagawa, G. (1996). Monte Carlo filter and smoother for non-Gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25.

Kong, A., Liu, J. S., and Wong, W. H. (1994). Sequential imputations and Bayesian missing data problems. *Journal of the American Statistical Association*, 89(425):278–288.

Lee, A. (2011). *On auxiliary variables and many-core architectures in computational statistics*. PhD thesis, Department of Statistics, University of Oxford, UK.

Lee, A., Singh, S. S., and Vihola, M. (2020). Coupled conditional backward sampling particle filter. *Annals of Statistics*, 48(5):3066–3089.

Li, Y., Wang, W., Deng, K., and Liu, J. S. (2022). Stratification and optimal resampling for sequential Monte Carlo. *Biometrika*, 109(1):181–194.

Lindsten, F., Jordan, M. I., and Schön, T. B. (2014). Particle Gibbs with ancestor sampling. *The Journal of Machine Learning Research*, 15(1):2145–2184.

Liu, J. S. (2001). *Monte Carlo Strategies in Scientific Computing*. Springer Series in Statistics. Springer.

Liu, J. S. and Chen, R. (1995). Blind deconvolution via sequential imputations. *Journal of the American Statistical Association*, 90(430):567–576.

Liu, J. S. and Chen, R. (1998). Sequential Monte Carlo methods for dynamic systems. *Journal of the American Statistical Association*, 93(443):1032–1044.

Martino, L., Elvira, V., and Louzada, F. (2016). Weighting a resampled particle in sequential Monte Carlo. In *2016 IEEE Statistical Signal Processing Workshop (SSP)*, pages 1–5. IEEE.

Medina-Aguayo, F. J. and Everitt, R. G. (2019). Revisiting the balance heuristic for estimating normalising constants. *arXiv preprint arXiv:1908.06514*.

Murray, L. M., Lee, A., and Jacob, P. E. (2016). Parallel resampling in the particle filter. *Journal of Computational and Graphical Statistics*, 25(3):789–805.

Pitt, M. K. and Shephard, N. (1999). Filtering via simulation: Auxiliary particle filters. *Journal of the American Statistical Association*, 94(446):590–599.

Rubin, D. B. (1987). Contribution to the discussion on 'the calculation of posterior distributions by data augmentation' by tanner, m. a. and wong, w. h. *Journal of the American Statistical Association*, 82(398):543–546.

Shestopaloff, A. and Doucet, A. (2019). Replica conditional sequential Monte Carlo. In *International Conference on Machine Learning*, pages 5749–5757. PMLR.

Stanton, C. and Beskos, A. (2024). Particle based inference for continuous-discrete state space models. *arXiv preprint arXiv:2407.15666*.

van den Boom, W., Jasra, A., De Iorio, M., Beskos, A., and Eriksson, J. G. (2022). Unbiased approximation of posteriors via coupled particle Markov chain Monte Carlo. *Statistics and Computing*, 32(36):1–19.

Veach, E. and Guibas, L. J. (1995). Optimally combining sampling techniques for Monte Carlo rendering. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, pages 419–428. ACM.

Walker, A. J. (1974). New fast method for generating discrete random numbers with arbitrary frequency distributions. *Electronics Letters*, 8(10):127–128.

Walker, A. J. (1977). An efficient method for generating discrete random variables with general distributions. *ACM Transactions on Mathematical Software*, 3(3):253–256.

Webber, R. J. (2019). Unifying sequential Monte Carlo with resampling matrices. *arXiv preprint arXiv:1903.12583*.

Whiteley, N. (2010). Contribution to the discussion on 'Particle Markov chain Monte Carlo methods' by Andrieu, C., Doucet, A., and Holenstein, R. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):306–307.

Whiteley, N., Andrieu, C., and Doucet, A. (2009). Particle Markov chain Monte Carlo for multiple change-point problems. Technical report 911, Department of Mathematics, Bristol University, UK.

Whiteley, N., Andrieu, C., and Doucet, A. (2010). Efficient Bayesian inference for switching state-space models using discrete particle Markov chain Monte Carlo methods. *arXiv e-prints*, page arXiv:1011.2437, 1011.2437.

Whiteley, N., Lee, A., and Heine, K. (2016). On the role of interaction in sequential Monte Carlo algorithms. *Bernoulli*, 22(1):494–529.

A On naïve implementations of conditional resampling

In this section, we illustrate that the two naïve strategies for implementing conditional resampling discussed in Section 4.3 of the main manuscript do not lead to a valid CSMC algorithm. Specifically, we will illustrate that conditional systematic and conditional residual-multinomial resampling implemented using the Strategies I/II (hereafter referred to as *naïve systematic resampling I/II* and *naïve residual-multinomial resampling I/II*) can induce a bias in the sense that the CSMC algorithm no longer leaves π_T invariant. Further, these naïve implementations can also worsen the autocorrelation of the Markov chain induced by the CSMC algorithm.

A.1 Binary state-space model

We illustrate the issue on a toy binary state-space model for which it is possible to evaluate the invariant distribution and autocorrelation of the Markov chain induced by a CSMC algorithm for small values of N and T. The model is as follows. For any $t \in [T]$, $\varepsilon, \delta \in [0,1]$ and any $x_t \in \mathsf{X} \coloneqq \{0,1\}$, let $M_1 = (1-\varepsilon)\delta_0 + \varepsilon\delta_1$, $M_{t+1}(\,\cdot\,|x_t) = (1-\varepsilon)\delta_{x_t} + \varepsilon\delta_{1-x_t}$ and $G_t(x_{t-1:t}) = G_t(x_t) = (1-\delta)\,\mathbf{1}\{x_t = 0\} + \delta\,\mathbf{1}\{x_t = 1\}$. Additionally, we let $N_1 = \ldots = N_T = N$ and $\lambda_1 = \mathrm{Unif}_{[N]}$.

As illustrated in Figure 1a, naïve conditional residual-multinomial resampling I suffers from substantial bias and increasing the number of particles or employing backward sampling reduces but does not remove this bias. Figure 1b demonstrates that the other naïve implementations also induce bias (albeit of a much smaller magnitude) with two exceptions:

1. Whenever the underlying resampling scheme is unbiased and $N = 2$, the naïve implementation of conditional resampling using Strategy II remains exact (in the sense that the Markov chain leaves π_T invariant) because it reduces to conditional multinomial resampling. We note that this holds generally, i.e., not just in this specific model.

2. Additionally, in this specific model, it can be shown that naïve conditional residual-multinomial resampling II reduces to conditional multinomial resampling (and is thus exact) whenever $|\delta - \frac{1}{2}| < 1/(4N - 6)$.

As illustrated in Figure 2, the fact that naïve conditional resampling II reduces to conditional multinomial resampling if $N = 2$ (and can still behave similarly to conditional multinomial resampling if $N > 2$) can also negatively impact the autocorrelation of the Markov chain induced by a CSMC algorithm, especially when the potential functions oscillate little.

A.2 Linear-Gaussian state-space model

One might speculate that the extent of the bias induced by naïve conditional resampling I was somehow due to the specific finite-state-space structure of the above example. Here, we illustrate that a similar bias is incurred in a simple univariate linear-Gaussian state-space model, i.e., if $M_1(x_1) = \mathrm{N}(x_1; 0, 1)$, $M_t(x_t|x_{t-1}) = \mathrm{N}(x_t; x_{t-1}, 1)$, for $t > 1$, and $G_t(x_{t-1:t}) = G_t(x_t) = \mathrm{N}(y_t; x_t, 1)$, for given observations $y_t \in \mathbb{R}$.

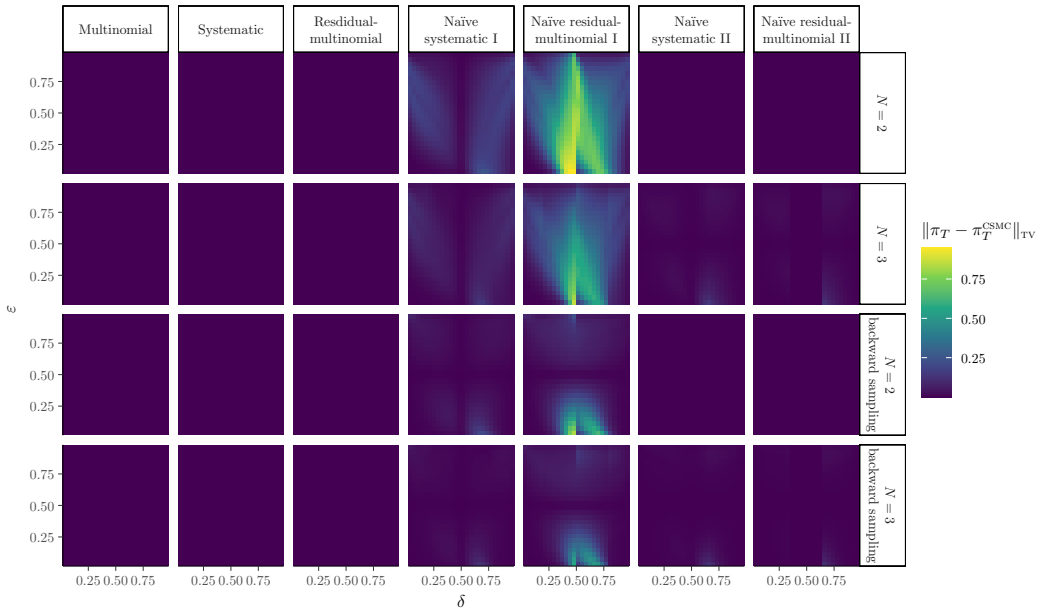
Figure 3 illustrates the bias induced in the approximation of the marginal posterior distribution at time $t = 2$ by naïve conditional resampling I in the case that $T = 4$ and $y_{1:4} = (0.75, 0.62, 1.91, 0.12)$ (the shown behaviour is typical for other observation sequences sampled from the model).

B Relationship with other frameworks

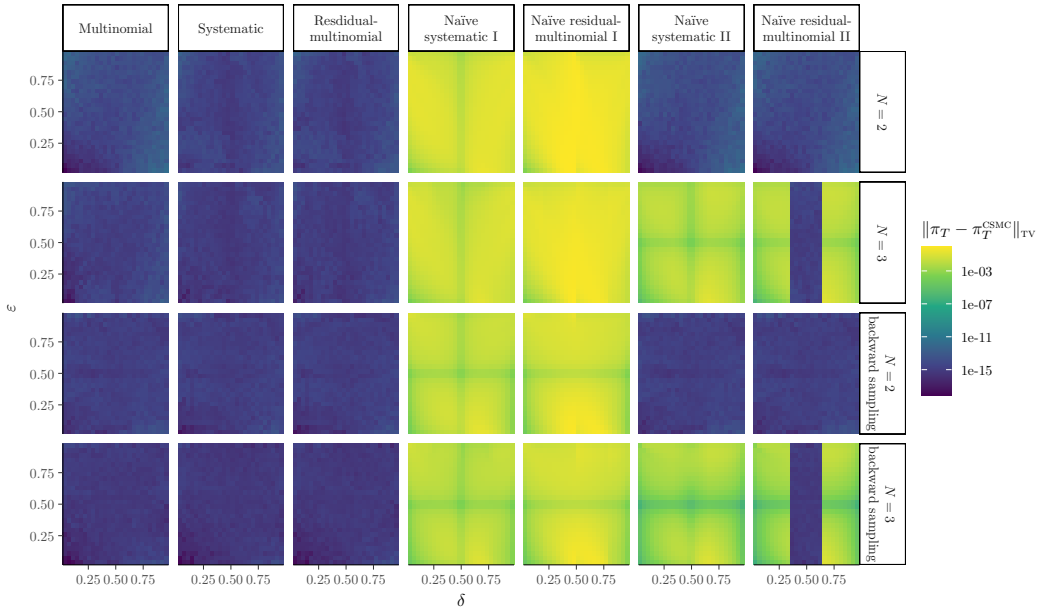
B.1 Relationship with (conditional) resampling based on randomly permuting or shifting the order of the ancestor indices

B.1.1 Random permutations

The framework from Andrieu et al. (2010) assumed, for simplicity, that the resampling schemes used within a (conditional) SMC algorithm are exchangeable. And although



(a) Bias on a linear scale.



(b) Bias on a logarithmic scale.

Figure 1: Total variation distance between the joint smoothing distribution, π_T, and the invariant distribution of the Markov chain induced by the CSMC algorithm, π_T^{CSMC}, in the binary state-space model with $T = 5$ time steps and $N_1 = \ldots = N_T = N$ particles. This figure illustrates that a naïve implementation of conditional resampling can induce bias.

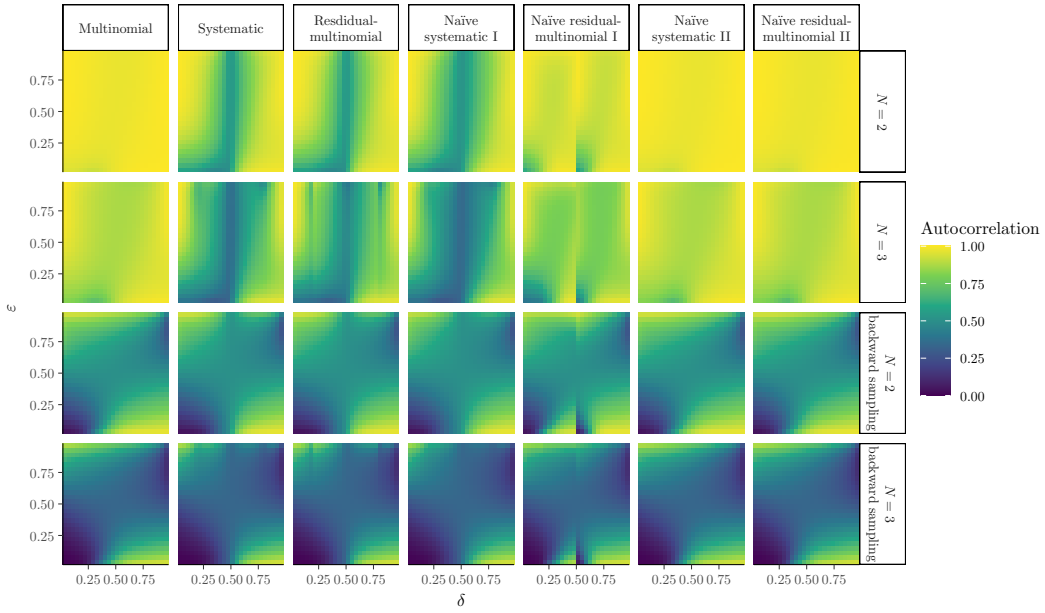


Figure 2: Lag-1 autocorrelation of the state at time $t = 1$ under the Markov chain induced by the CSMC algorithm for the same setting as in Figure 1b. This figure illustrates that naïve implementations of conditional resampling can induce a substantially worse autocorrelation than their exact counterparts (except for naïve conditional systematic resampling I); and for nearly constant potential functions ($\delta \approx 0.5$) the difference remains even if backward sampling is used.

many popular resampling schemes (like systematic resampling) are not exchangeable, Andrieu et al. (2010) noted that any resampling schemes can be made exchangeable by randomly permuting the order of the ancestor indices after they have been sampled. This idea was used to derive a conditional version of residual-multinomial resampling in Chopin and Singh (2015, Algorithm 2) in which K is always set to 1. In this section, we show that unless an implementation of conditional resampling with $K = 1$, say, is required, the use such random permutations is unnecessary (and only adds computational cost).

Formally, let $\langle \varrho, \kappa \rangle$ be some balanced resampling scheme which may not be exchangeable. We can then derive an exchangeable resampling scheme $\langle \rho, \lambda \rangle$ by setting

$$\rho(\mathbf{a}, u) = \sum_{\mathbf{b} \in [N]^M} \varrho(\mathbf{b}, u) \sum_{\sigma \in \mathsf{S}_M} \mathrm{Unif}_{\mathsf{S}_M}(\sigma) \delta_{\mathbf{b}^\sigma}(\mathbf{a}),$$

where for any $\mathbf{b} \coloneqq b^{1:M}$ and any permutation $\sigma \in \mathsf{S}_M \coloneqq \{\tau \colon [M] \to [M] \mid \tau \text{ is bijective}\}$, we write $\mathbf{b}^\sigma \coloneqq (b^{\sigma(1)}, \ldots, b^{\sigma(M)})$; as well as by setting $\lambda(m, u|n) = \mathrm{Unif}_{[M]}(m) \rho^m(u|n)$,

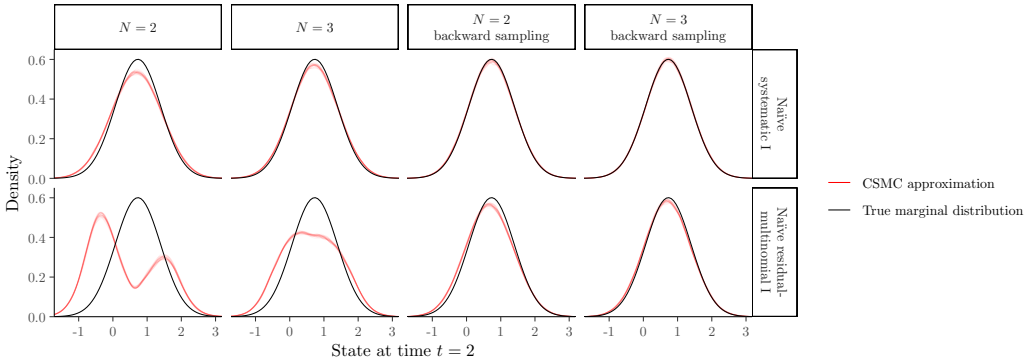


Figure 3: Approximation of the marginal smoothing distribution at time $t = 2$ in the linear-Gaussian state-space model. The red lines represent 25 independent repetitions of each algorithm using 10^6 iterations each (after burnin).

for any $n \in [N]$; then

$$\begin{aligned}\phi(m,n,u) &= MW^n \frac{\lambda(m,u|n)}{\rho^m(n,u)} \\ &= W^n \frac{1}{\frac{1}{\#\mathsf{S}_M}\sum_{\sigma\in\mathsf{S}_M}\sum_{l=1}^M \mathbf{1}\{\sigma(m)=l\}\varrho^l(n)} = \frac{MW^n}{\sum_{l=1}^M \varrho^l(n)}.\end{aligned}$$

Thus, application of the random permutation does not change the law of the resampled measure $\tilde{\pi}$.

Algorithm 28 resampling with random permutation

1: Sample $(\mathbf{B}, U) = (B^{1:M}, U) \sim \varrho$.
2: Sample $\Sigma \sim \mathrm{Unif}_{\mathsf{S}_M}$ and set $\mathbf{A} := \mathbf{B}^\Sigma$.
3: Set $\widetilde{X}^m := X^{A^m}$ and $\tilde{w}^m := W^{A^m} / \sum_{m=1}^M \varrho^m(A^m)$, for $m \in [M]$.

Comparing Algorithm 28 with Algorithm 1, we see that randomly permuting the the order of the ancestor indices only adds computational cost Specifically, the former requires the additional Line 2.

Sampling the index K within the conditional version of $\langle \rho, \lambda \rangle$ becomes simpler compared to $\langle \varrho, \kappa \rangle$ since the distribution of K is now uniform under $\lambda(\,\cdot\,|a)$. However, conditional resampling also requires sampling from $\rho^k(u|a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k,u)\delta_a(a^k)$ and the

random permutation makes the latter more involved:

$$\begin{aligned}
&\rho^k(u|a^k)\rho^{-k}(\mathbf{a}^{-k}|a^k,u)\delta_a(a^k)\\
&\quad= \frac{\rho(\mathbf{a},u)\,\mathbf{1}\{a^k=a\}}{\rho^k(a)}\\
&\quad= \frac{\sum_{\mathbf{b}\in[N]^M}\varrho(\mathbf{b},u)\sum_{\sigma\in\mathsf{S}_M}\mathrm{Unif}_{\mathsf{S}_M}(\sigma)\delta_{\mathbf{b}^\sigma}(\mathbf{a})\,\mathbf{1}\{a^k=a\}}{\frac{1}{M}\sum_{m=1}^M\varrho^m(a)}\\
&\quad= \sum_{\mathbf{b}\in[N]^M}\sum_{l=1}^M\frac{\varrho^l(a)}{\sum_{m=1}^M\varrho^m(a)}\varrho^l(u|a)\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l,u)\sum_{\sigma\in\mathsf{S}_M^k(l)}\mathrm{Unif}_{\mathsf{S}_M^k(l)}(\sigma)\delta_{\mathbf{b}^\sigma}(\mathbf{a})\\
&\quad= \sum_{\mathbf{b}\in[N]^M}\sum_{l=1}^M\kappa(l,u|a)\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l,u)\sum_{\sigma\in\mathsf{S}_M^k(l)}\mathrm{Unif}_{\mathsf{S}_M^k(l)}(\sigma)\delta_{\mathbf{b}^\sigma}(\mathbf{a}),
\end{aligned}$$

where $\mathsf{S}_M^k(l) \coloneqq \{\sigma\in\mathsf{S}_M \mid \sigma(k)=l\}$ which, in particular, means that in the last two lines, $\sigma\in\mathsf{S}_M^k(l)$ implies that $b^l = b^{\sigma(k)} = a^{\sigma^{-1}(l)} = a^k$.

Algorithm 29 conditional resampling with random permutation

Require: $a\in[N]$ with $W^a>0$.
1: Sample $(L,U,\mathbf{B})\sim\kappa(l,u|a)\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l,u)$.
2: Sample $K\sim\mathrm{Unif}_{[M]}$ as well as $\Sigma\sim\mathrm{Unif}_{\mathsf{S}_M^K(L)}$ and set $\mathbf{A}\coloneqq\mathbf{B}^\Sigma$.
3: Set $\widetilde{X}^m\coloneqq X^{A^m}$ and $\widetilde{w}^m\coloneqq W^{A^m}/\sum_{m=1}^M\varrho^m(A^m)$, for $m\in[M]$.

Again by comparing Algorithm 29 with Algorithm 2, we see that the random permutation incurs the additional Line 2.

In summary, our work shows that randomly permuting the order of the ancestor indices is not needed (and only adds unnecessary computational cost) unless one wishes to exploit Proposition 5 to implement conditional resampling in a way that always sets $K\coloneqq 1$ or $K\coloneqq M$, say, in Line 2.

B.1.2 Random cyclical shifts

All the developments from the previous section remain valid if we replace the random permutations $\sigma\in\mathsf{S}_M$ by the subset of S_M which correspond to random cyclical shifts $\sigma\in\mathsf{C}_M\coloneqq\{\tau\colon[M]\to[M]\mid\exists\, s\in[M]\;\forall\, m\in[M]:\tau(m)=m+s \mod M\}$ (and if we similarly replace $\mathsf{S}_M^k(l)$ by $\mathsf{C}_M^k(l)\coloneqq\{\sigma\in\mathsf{C}_M\mid\sigma(k)=l\}$). In particular, note that the resampling scheme $\langle\rho,\lambda\rangle$ induced by applying such random cyclical shifts to the order of the ancestor indices generated by ϱ still satisfies the assumption of Proposition 5.

Algorithm 30 resampling with random shift

1: Sample $(\mathbf{B},U)=(B^{1:M},U)\sim\varrho$.
2: Sample $\Sigma\sim\mathrm{Unif}_{\mathsf{C}_M}$ and set $\mathbf{A}\coloneqq\mathbf{B}^\Sigma$.
3: Set $\widetilde{X}^m\coloneqq X^{A^m}$ and $\widetilde{w}^m\coloneqq W^{A^m}/\sum_{m=1}^M\varrho^m(A^m)$, for $m\in[M]$.

Algorithm 31 conditional resampling with random shift

Require: $a \in [N]$ with $W^a > 0$.

1: Sample $(L, U, \mathbf{B}) \sim \kappa(l, u|a)\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l, u)$.
2: Sample $K \sim \text{Unif}_{[M]}$ as well as $\Sigma \sim \text{Unif}_{\mathsf{C}_M^K(L)}$ and set $\mathbf{A} \coloneqq \mathbf{B}^\Sigma$.
3: Set $\widetilde{X}^m \coloneqq X^{A^m}$ and $\widetilde{w}^m \coloneqq W^{A^m} / \sum_{m=1}^M \varrho^m(A^m)$, for $m \in [M]$.

Such random cyclical shifts were employed in Chopin and Singh (2015, Algorithm 4) to derive a conditional version of systematic resampling in which K can always be set to 1. Specifically, Chopin and Singh (2015, Algorithm 4) can be derived from Algorithm 31 by taking $\langle \varrho, \kappa \rangle$ to be the systematic resampling scheme and $a = 1$ as follows:

- in Line 1 of Algorithm 31, sample

$$U \sim \frac{\lfloor MW^1 \rfloor}{MW^1} \text{Unif}_{[0,1]}(u) + \left(1 - \frac{\lfloor MW^1 \rfloor}{MW^1}\right) \text{Unif}_{(0, MW^1 - \lfloor MW^1 \rfloor]}(u), \tag{13}$$

  which is justified by the fact that $\kappa(u|a)$ in

$$\begin{aligned}\kappa(l, u|a) &= \kappa(u|a)\kappa(l|u, a) \\ &= \left[\sum_{m=1}^M \frac{\text{vol}(I^m(a))}{MW^a} \text{Unif}_{I^m(a)}(u)\right] \text{Unif}_{\{m \in [M] | u \in I^m(a)\}}(l),\end{aligned}$$

  simplifies to the r.h.s. in (13) due to $a = 1$;

- also in Line 1 of Algorithm 31, set $B^m \coloneqq F^{-1}([U + m - 1]/M)$, for $m \in [M]$, using Algorithm 7 with $U^1 = \ldots = U^M = U$ and simply set $L \coloneqq 1$ both of which are justified by the fact that $a = 1$ and that given $u \sim \kappa(u|a)$,

$$\delta_a(b^l)\varrho^{-l}(\mathbf{b}^{-l}|b^l, u) = \prod_{m=1}^M \delta_{F^{-1}([u+m-1]/M)}(b^m)$$

  does not depend on l;

- in Line 2 of Algorithm 31, appeal to Proposition 5 to set $K \coloneqq 1$.

Furthermore, Karppinen et al. (2023, Algorithms 5 & 6) introduced conditional versions of killing resampling and of systematic resampling with weight re-ordering based on such random cyclical shifts which could likewise be justified as instances of Algorithm 31 in which $\langle \varrho, \kappa \rangle$ is killing resampling or systematic resampling with weight re-ordering, respectively.

Our work shows that such random cyclical shifts are only necessary if one wishes to exploit Proposition 5 to implement conditional resampling in a way that always sets $K \coloneqq 1$ or $K \coloneqq M$, say, in Line 2.

B.2 Relationship with αSMC resampling/resamping matrices

The αSMC framework from Whiteley et al. (2016) resampling provides a matrix representation of resampling schemes for which $\mathsf{U} := \emptyset$ and the resampling distribution factorises as

$$\rho(\mathbf{a}) = \prod_{m=1}^{M} \rho^m(a^m). \tag{14}$$

Let $\alpha = (\alpha^{m,n}) \in [0,\infty)^{M\times N}$ be some $\mathcal{F}$-measurable *resampling matrix* which is such that each row defines a probability measure on $[N]$, i.e., such that $\sum_{n=1}^{N} \alpha^{m,n} = 1$, for any $m \in [M]$. We will also assume that

$$\sum_{m=1}^{M} \alpha^{m,n} > 0, \tag{15}$$

for any $n \in [N]$ with $W^n > 0$.

Then αSMC represents $\rho^m(n)$ as

$$\rho^m(n) := \mathrm{Cat}\left(n; \left(\frac{\alpha^{m,k}W^k}{\sum_{l=1}^{N}\alpha^{m,l}W^l}\right)_{k=1}^{N}\right),$$

which is well defined by (15) (and the usual convention $\frac{0}{0} = 0$).

Use of such resampling matrices goes back to at least Hu et al. (2008)

To embed αSMC resampling within our framework, set

$$\lambda(m|n) := \frac{\alpha^{m,n}}{\sum_{l=1}^{M}\alpha^{l,n}},$$

whenever $W^n > 0$, so that

$$\phi(m,n) = MW^n\frac{\lambda(m|n)}{\rho^m(n)} = M\sum_{l=1}^{N}\frac{\alpha^{m,l}}{\sum_{k=1}^{M}\alpha^{k,l}}W^l.$$

Then the αSMC resampling scheme $\langle\rho,\lambda\rangle$ is not necessarily balanced but still properly weighted and persistent.

The factorisation (14) holds for most of the elementary resampling schemes from Section 3.1, including trivial resampling, i.e., not resampling (recall that as explained in Remark 3, whether or not we explicitly include the auxiliary variable U into the space is immaterial for balanced resampling schemes). And since the resampling matrix α is allowed to be some (measurable) function of the particle system, αSMC even certain kinds of ESS-adaptive resampling. Additionally, a conditional version of αSMC resampling was derived in Huggins and Roy (2019).

However, the αSMC framework does not cover resampling distributions which violate the factorisation from (14), such as the widely used (adaptive) systematic resampling scheme or more 'exotic' schemes like chopthin resampling. This shows that our framework provides a strict generalisation of αSMC.

C Proofs for Section 2

Proof (of Proposition 5). Since $\langle \rho, \lambda \rangle$ is balanced, $\phi(m, n, u)$ is constant in u so that

$$f^b(\mathbf{a}) \coloneqq \sum_{l=1}^{M} \frac{\phi(l, a^l, u)}{\sum_{m=1}^{M} \phi(m, a^m, u)} \mathbf{1}\{a^l = b\} = \sum_{l=1}^{M} \frac{W^{a^l} / \sum_{i=1}^{M} \rho^i(a^l)}{\sum_{m=1}^{M} W^{a^m} / \sum_{j=1}^{M} \rho^j(a^m)} \mathbf{1}\{a^l = b\},$$

is well defined and symmetric in all its M arguments. Let $\mu(k, a, \mathbf{a}, \mathrm{d}u, b, l)$ be the extended distribution from the proof of Proposition 4 and let $\mu'(k, a, \mathbf{a}, \mathrm{d}u, b, l)$ be the same distribution but with $\lambda(k, \mathrm{d}u|a) = \lambda(k|a)\rho^k(\mathrm{d}u|a)$ replaced by $\lambda'(k|a)\rho^k(\mathrm{d}u|a)$. Then the marginal distribution $\mu(b)$ of b under both these extended distributions is the same:

$$\mu'(b) = \sum_{k=1}^{M} W^a \lambda'(k|a)\, \mathbb{E}_{k,a}[f^b(\mathbf{A})] = \mathbb{E}_{1,a}[f^b(\mathbf{A})] = \sum_{k=1}^{M} W^a \lambda(k|a)\, \mathbb{E}_{k,a}[f^b(\mathbf{A})] = \mu(b).$$

Thus, the proof of Proposition 4 is not affected by the replacement of $\lambda(k|a)$ by $\lambda'(k|a)$. □

D Proofs for Section 4

D.1 CSMC with backward or ancestor sampling

We again assume that $N_t \geq 2$ and make the following assumption on the mutation kernels to ensure that the transition densities $m_t(\,\cdot\,|x) \coloneqq \mathrm{d}M_t(\,\cdot\,|x)/\mathrm{d}\nu$ are well defined for any $x \in \mathsf{X}$ and $t \in [T]$.

Assumption 1. *There exists a measure ν on X such that $M_t(\,\cdot\,|x) \ll \nu$, for any $x \in \mathsf{X}$, is absolutely continuous w.r.t. nu, for all $x \in \mathsf{X}$ and $t \in [T]$.*

Assumption 1 guarantees the existence of the following *backward kernel*, for any $k \in [N_t]$, any $t < T$ and with the notation $\mathbf{x}_t \coloneqq x^{1:N_t}$:

$$B_t(k|\mathbf{x}_t, x_{t+1}; \mathbf{W}_t) \coloneqq \frac{W_t^k G_{t+1}(x_t^k, x_{t+1}) m_{t+1}(x_{t+1}|x_t^k)}{\sum_{n=1}^{N_t} W_t^n G_{t+1}(x_t^n, x_{t+1}) m_{t+1}(x_{t+1}|x_t^n)}. \tag{16}$$

Algorithm 32 CSMC with backward sampling

Require: $x_{1:T} \in \mathsf{X}^T$ with $\pi_T(x_{1:T}) > 0$.

for $t = 1, \ldots, T$ **do**
  if $t = 1$ **then**
    Sample $K_1 \sim \lambda_1$.
    Set $\widetilde{w}_{t-1}^n \coloneqq \lambda_1(n)$, for $n \in [N_1]$.
  else
    Sample $(K_t, \mathbf{A}_{t-1}^{-K_t}, U_{t-1}) \sim \lambda_t(k_t, u_{t-1} | K_{t-1}; \mathbf{W}_{t-1}) \rho_{t-1}^{-k_t}(\mathbf{a}_{t-1}^{-k_t} | K_{t-1}, u_{t-1}; \mathbf{W}_{t-1})$.
    Set $A_{t-1}^{K_t} \coloneqq K_{t-1}$.
    Set $\widetilde{w}_{t-1}^n \coloneqq \phi_t(n, A_{t-1}^n, U_{t-1}; \mathbf{W}_{t-1})/N_t$ via (12), and $\widetilde{X}_{t-1}^n \coloneqq X_{t-1}^{A_{t-1}^n}$, for $n \in [N_t]$.
  Set $X_t^{K_t} \coloneqq x_t$, and sample $X_t^n \sim M_t(\cdot | \widetilde{X}_{t-1}^n)$, for $n \in [N_t] \setminus \{K_t\}$.
  Set $w_t^n \coloneqq \widetilde{w}_{t-1}^n G_t(\widetilde{X}_{t-1}^n, X_t^n)$, and $W_t^n \coloneqq w_t^n / \sum_{m=1}^{N_t} w_t^m$, for $n \in [N_t]$.
Sample $L_T \sim \mathrm{Cat}(\mathbf{W}_T)$ and sample $L_t \sim B_t(\cdot | \mathbf{X}_t, X_{t+1}^{L_{t+1}}; \mathbf{W}_t)$, for $t = T-1, \ldots, 1$.
Return $X'_{1:T} \coloneqq (X_1^{L_1}, \ldots, X_T^{L_T})$.

Algorithm 33 CSMC with ancestor sampling

Require: $x_{1:T} \in \mathsf{X}^T$ with $\pi_T(x_{1:T}) > 0$.

for $t = 1, \ldots, T$ **do**
  if $t = 1$ **then**
    Sample $K_1 \sim \lambda_1$.
    Set $\widetilde{w}_{t-1}^n \coloneqq \lambda_1(n)$, for $n \in [N_1]$.
  else
    Sample $(K_t, U_{t-1}) \sim \lambda_t(k_t, u_{t-1} | K_{t-1}; \mathbf{W}_{t-1})$.
    Sample $\mathbf{A}_{t-1} \sim B_{t-1}(a_{t-1}^{K_t} | \mathbf{X}_{t-1}, x_t; \mathbf{W}_{t-1}) \rho_{t-1}^{-K_t}(\mathbf{a}_{t-1}^{-K_t} | A_{t-1}^{K_t}, U_{t-1}; \mathbf{W}_{t-1})$,
    Set $\widetilde{w}_{t-1}^n \coloneqq \phi_t(n, A_{t-1}^n, U_{t-1}; \mathbf{W}_{t-1})/N_t$ via (12), and $\widetilde{X}_{t-1}^n \coloneqq X_{t-1}^{A_{t-1}^n}$, for $n \in [N_t]$.
  Set $X_t^{K_t} \coloneqq x_t$, and sample $X_t^n \sim M_t(\cdot | \widetilde{X}_{t-1}^n)$ for $n \in [N_t] \setminus \{K_t\}$.
  Set $w_t^n \coloneqq \widetilde{w}_{t-1}^n G_t(\widetilde{X}_{t-1}^n, X_t^n)$, and $W_t^n \coloneqq w_t^n / \sum_{m=1}^{N_t} w_t^m$, for $n \in [N_t]$.
Sample $L_T \sim \mathrm{Cat}(\mathbf{W}_T)$ and set $L_t \coloneqq A_t^{L_{t+1}}$ for $t = T-1, \ldots, 1$.
Return $X'_{1:T} \coloneqq (X_1^{L_1}, \ldots, X_T^{L_T})$.

The following result—proved in Appendix D.5—extends Proposition 8.

Proposition 9. *Assume that the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are properly weighted and persistent for any $t \in [T] \setminus \{1\}$ and that P is the Markov kernel induced by Algorithm 32 or Algorithm 33. Then P is π_T-invariant.*

D.2 Extended state-space construction

Throughout this section, we write $\mathbf{X}_t \coloneqq X_t^{1:N_t}$ as well as $\mathbf{Z}_t \coloneqq (\mathbf{X}_t, \mathbf{A}_{t-1}, U_{t-1})$, for $t > 1$ and $\mathbf{Z}_1 \coloneqq \mathbf{X}_1$. The weights $\mathbf{W}_t = W_t^{1:N_t}$ then depend on $\mathbf{Z}_{1:T}$ although we do not make

this explicit to simplify the notation. The following distributions are well defined if the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are all properly weighted and persistent.

1. The law of all the random variables generated by the SMC algorithm (Algorithm 26) is then

$$\mathrm{smc}_T(\mathrm{d}\mathbf{z}_{1:T}) \coloneqq \left[\prod_{n=1}^{N_1} M_1(\mathrm{d}x_1^n)\right] \prod_{t=2}^{T}\left[\rho_{t-1}(\mathrm{d}[\mathbf{a}_{t-1}, u_{t-1}]; \mathbf{W}_{t-1}) \prod_{n=1}^{N_t} M_t(\mathrm{d}x_t^n | x_{t-1}^{a_{t-1}^n})\right].$$

2. Similarly, the law of all the random variables generated by the CSMC algorithm with backward sampling (Algorithm 32)—under Assumption 1—is

$$\mathrm{csmc}_T(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T})\mathrm{back}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}),$$

where the first term is the law of all the random variables generated by Lines 1–10:

$$\begin{aligned}
&\mathrm{csmc}_T(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T}) \\
&\quad\coloneqq \left[\lambda_1(\mathrm{d}k_1)\delta_{x_1}(\mathrm{d}x_1^{k_1}) \prod_{\substack{n=1 \\ n\neq k_1}}^{N_1} M_1(\mathrm{d}x_1^n)\right] \prod_{t=2}^{T}\left[\lambda_t(\mathrm{d}[k_t, u_{t-1}]|k_{t-1}, \mathbf{W}_{t-1})\right. \\
&\qquad \left.\times \delta_{k_{t-1}}(\mathrm{d}a_{t-1}^{k_t})\rho_{t-1}^{-k_t}(\mathrm{d}\mathbf{a}_{t-1}^{-k_t}|a_{t-1}^{k_t}, u_{t-1}; \mathbf{W}_{t-1})\delta_{x_t}(\mathrm{d}x_t^{k_t}) \prod_{\substack{n=1 \\ n\neq k_t}}^{N_t} M_t(\mathrm{d}x_t^n | x_{t-1}^{a_{t-1}^n})\right];
\end{aligned}$$

and the second term is the law of all the random variables generated in Lines 11–12:

$$\mathrm{back}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \coloneqq W_T^{l_T}\left[\prod_{t=1}^{T-1} B_t(\mathrm{d}l_t|\mathbf{x}_t, x_{t+1}^{l_{t+1}}; \mathbf{W}_t)\right] \prod_{t=1}^{T} \delta_{x_t^{l_t}}(\mathrm{d}x'_t).$$

3. Finally, the law of all the random variables generated by the CSMC algorithm with ancestor sampling (Algorithm 33)—again under Assumption 1— is

$$\mathrm{csmc}_T^*(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T})\mathrm{anc}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}),$$

where the first term is the law of all the random variables generated by Lines 1–10:

$$\begin{aligned}
&\mathrm{csmc}_T^*(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T}) \\
&\quad\coloneqq \left[\lambda_1(\mathrm{d}k_1)\delta_{x_1}(\mathrm{d}x_1^{k_1}) \prod_{\substack{n=1 \\ n\neq k_1}}^{N_1} M_1(\mathrm{d}x_1^n)\right] \\
&\qquad \times \prod_{t=2}^{T}\left[\lambda_t(\mathrm{d}[k_t, u_{t-1}]|k_{t-1}, \mathbf{W}_{t-1})B_{t-1}(\mathrm{d}a_{t-1}^{k_t}|\mathbf{x}_{t-1}, x_t; \mathbf{W}_{t-1})\right. \\
&\qquad\quad \left.\times \rho_{t-1}^{-k_t}(\mathrm{d}\mathbf{a}_{t-1}^{-k_t}|a_{t-1}^{k_t}, u_{t-1}; \mathbf{W}_{t-1})\delta_{x_t}(\mathrm{d}x_t^{k_t}) \prod_{\substack{n=1 \\ n\neq k_t}}^{N_t} M_t(\mathrm{d}x_t^n | x_{t-1}^{a_{t-1}^n})\right];
\end{aligned}$$

and the second term is the law of all the random variables generated in Lines 11–12:

$$\mathrm{anc}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \coloneqq W_T^{l_T}\left[\prod_{t=1}^{T-1} \delta_{a_t^{l_{t+1}}}(\mathrm{d}l_t)\right] \prod_{t=1}^{T} \delta_{x_t^{l_t}}(\mathrm{d}x'_t).$$

In the following, we use the convention that $\mu(\mathrm{d}x)P(\mathrm{d}y|x) = [\mu \otimes P](\mathrm{d}[x,y])$ and with some abuse of notation regarding the order of the arguments. We also make the dependence of $\widehat{\mathcal{Z}}_T = \widehat{\mathcal{Z}}_T(\mathbf{Z}_{1:T})$ on $\mathbf{Z}_{1:T}$ explicit in the notation.

Lemma 1. *Assume that the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are properly weighted and persistent for all $t \in [T] \setminus \{1\}$. Then:*

1. *The Radon–Nikodym derivative*

$$\frac{\mathrm{d}[\gamma_T \otimes \mathrm{csmc}_T]}{\mathrm{d}[\mathrm{smc}_T \otimes \mathrm{anc}_T]}(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T}) = \widehat{\mathcal{Z}}_T(\mathbf{z}_{1:T})\, \mathbf{1}\{(x_1^{k_1}, \ldots, x_T^{k_T}) = x_{1:T}, (a_1^{k_2}, \ldots, a_{T-1}^{k_T}) = k_{1:T-1}\},$$

   is well defined for $[\mathrm{smc}_T \otimes \mathrm{anc}_T]$-almost all $(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T})$.

2. *The Radon–Nikodym derivative*

$$\frac{\mathrm{d}[\gamma_T \otimes \mathrm{csmc}_T^*]}{\mathrm{d}[\mathrm{smc}_T \otimes \mathrm{back}_T]}(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T}) = \widehat{\mathcal{Z}}_T(\mathbf{z}_{1:T})\, \mathbf{1}\{(x_1^{k_1}, \ldots, x_T^{k_T}) = x_{1:T}\},$$

   is well defined for $[\mathrm{smc}_T \otimes \mathrm{back}_T]$-almost all $(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T})$ if additionally Assumption 1 holds.

Proof. For Part 1, since the resampling schemes $\langle \rho_{t-1}, \lambda_t \rangle$ are properly weighted and persistent, for $[\mathrm{smc}_T \otimes \mathrm{anc}_T]$-almost all $(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T})$, we have

$$\begin{aligned}
&\frac{\mathrm{d}[\gamma_T \otimes \mathrm{csmc}_T]}{\mathrm{d}[\mathrm{smc}_T \otimes \mathrm{anc}_T]}(\mathbf{z}_{1:T}, k_{1:T}, x_{1:T}) \\
&\quad = \lambda_1(k_1)\left[\prod_{t=1}^{T} G_t(x_{t-1}^{k_{t-1}}, x_t^{k_t})\right](W_T^{k_T})^{-1} \\
&\qquad \times \left[\prod_{t=2}^{T} \frac{\lambda_t(k_t, u_{t-1}|k_{t-1}, \mathbf{W}_{t-1})\rho_{t-1}^{-k_t}(\mathbf{a}_{t-1}^{-k_t}|a_{t-1}^{k_t}, u_{t-1}; \mathbf{W}_{t-1})}{\rho_{t-1}(\mathbf{a}_{t-1}, u_{t-1}; \mathbf{W}_{t-1})}\right] \\
&\qquad \times \mathbf{1}\{(x_1^{k_1}, \ldots, x_T^{k_T}) = x_{1:T}, (a_1^{k_2}, \ldots, a_{T-1}^{k_T}) = k_{1:T-1}\} \\
&\quad = \lambda_1(k_1)G_1(x_1^{k_1})(W_T^{k_T})^{-1}\left[\prod_{t=2}^{T} G_t(x_{t-1}^{k_{t-1}}, x_t^{k_t})\frac{\lambda_t(k_t, u_{t-1}|k_{t-1}, \mathbf{W}_{t-1})}{\rho_{t-1}^{k_t}(a_{t-1}^{k_t}, u_{t-1}; \mathbf{W}_{t-1})}\right] \\
&\qquad \times \mathbf{1}\{(x_1^{k_1}, \ldots, x_T^{k_T}) = x_{1:T}, (a_1^{k_2}, \ldots, a_{T-1}^{k_T}) = k_{1:T-1}\} \\
&\quad = \widehat{\mathcal{Z}}_T(\mathbf{z}_{1:T})\, \mathbf{1}\{(x_1^{k_1}, \ldots, x_T^{k_T}) = x_{1:T}, (a_1^{k_2}, \ldots, a_{T-1}^{k_T}) = k_{1:T-1}\},
\end{aligned}$$

where the last identity uses that $\lambda_1(k_1)G_1(x_1^{k_1}) = w_1^{k_1}$ and that the terms within the brackets in the penultimate line are equal to $w_t^{k_t}/W_{t-1}^{k_{t-1}}$.

Part 2 follows analogously using the definition of the backward kernels in (16). □

D.3 Proof of Proposition 7

The proof slightly extends the 'SMC as importance sampling' argument from Andrieu et al. (2010, Appendix B1) to our setting.

By Part 1 of Lemma 1, we have $\gamma_T \otimes \mathrm{csmc}_T \ll \mathrm{smc}_T \otimes \mathrm{anc}_T$. The proof then follows by a change of measure and the tower property of conditional expectation:

$$\begin{aligned}
\gamma_T(f) &= \int f(x_{1:T})[\gamma_T \otimes \mathrm{csmc}_T](\mathrm{d}[\mathbf{z}_{1:T}, k_{1:T}, x_{1:T}]) \\
&= \mathbb{E}\left[f(X_{1:T})\frac{\mathrm{d}[\gamma_T \otimes \mathrm{csmc}_T]}{\mathrm{d}[\mathrm{smc}_T \otimes \mathrm{anc}_T]}(\mathbf{Z}_{1:T}, K_{1:T}, X_{1:T})\right] \\
&= \mathbb{E}\left[\mathbb{E}\left(f(X_{1:T})\frac{\mathrm{d}[\gamma_T \otimes \mathrm{csmc}_T]}{\mathrm{d}[\mathrm{smc}_T \otimes \mathrm{anc}_T]}(\mathbf{Z}_{1:T}, K_{1:T}, X_{1:T})\middle|\mathbf{Z}_{1:T}\right)\right] \\
&= \mathbb{E}\left[\widehat{\mathcal{Z}}_T(\mathbf{Z}_{1:T})\sum_{n=1}^{N_T} W_T^n f(X_{1:T}^{(n)})\right] \\
&= \mathbb{E}[\hat{\gamma}_T(f)],
\end{aligned}$$

where the expectations in the first three lines are w.r.t. $(\mathbf{Z}_{1:T}, K_{1:T}, X_{1:T}) \sim \mathrm{smc}_T \otimes \mathrm{anc}_T$ and the expectations in the last two lines are w.r.t. $\mathbf{Z}_{1:T} \sim \mathrm{smc}_T$. □

D.4 Proof of Proposition 8

The first part of Lemma 1, gives

$$\begin{aligned}
&\pi_T(\mathrm{d}x_{1:T})\mathrm{csmc}_T(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T})\mathrm{anc}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \\
&\quad = \widehat{\mathcal{Z}}_T(\mathbf{z}_{1:T})\mathrm{smc}_T(\mathrm{d}\mathbf{z}_{1:T})\mathrm{anc}_T(\mathrm{d}[k_{1:T}, x_{1:T}]|\mathbf{z}_{1:T})\mathrm{anc}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \\
&\quad = \pi_T(\mathrm{d}x'_{1:T})\mathrm{csmc}_T(\mathrm{d}[l_{1:T}, \mathbf{z}_{1:T}]|x'_{1:T})\mathrm{anc}_T(\mathrm{d}[k_{1:T}, x_{1:T}]|\mathbf{z}_{1:T}).
\end{aligned}$$

This completes the proof. □

D.5 Proof of Proposition 9

The proof could be established in a number of ways, e.g., using partially-collapsed Gibbs sampling arguments (Whiteley, 2010; Lindsten et al., 2014). Here, we use the proof technique from Finke (2015) which is convenient in the sense that it simultaneously guarantees the validity of backward and ancestor sampling.

Specifically, appealing to both parts of Lemma 1,

$$\begin{aligned}
&\pi_T(\mathrm{d}x_{1:T})\mathrm{csmc}_T(\mathrm{d}[k_{1:T}, \mathbf{z}_{1:T}]|x_{1:T})\mathrm{back}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \\
&\quad = \widehat{\mathcal{Z}}_T(\mathbf{z}_{1:T})\mathrm{smc}_T(\mathrm{d}\mathbf{z}_{1:T})\mathrm{anc}_T(\mathrm{d}[k_{1:T}, x_{1:T}]|\mathbf{z}_{1:T})\mathrm{back}_T(\mathrm{d}[l_{1:T}, x'_{1:T}]|\mathbf{z}_{1:T}) \\
&\quad = \pi_T(\mathrm{d}x'_{1:T})\mathrm{csmc}^*_T(\mathrm{d}[l_{1:T}, \mathbf{z}_{1:T}]|x'_{1:T})\mathrm{anc}_T(\mathrm{d}[k_{1:T}, x_{1:T}]|\mathbf{z}_{1:T}).
\end{aligned}$$

This completes the proof in the case of backward or ancestor sampling. □